

ELECTRONICS SHOP INVENTORY MANAGEMENT SYSTEM
A PROJECT SUBMITTED TO



BHAKTA KAVI NARSINH MEHTA UNIVERSITY
FOR THE DEGREE
OF
BACHLOR OF COMPUTER APPLICATION
IN COMPUTER SCIENCE

Student Name

JETHWA MANSI AMITKUMAR

(ENROLLMENT NUMBER: 20222011080)

Project Guide

Prof. Jaydip Rathod

Associate Professor

Department of Computer Science

Shri V. J. Modha College of Information & Technology- Porbandar

Table of Contents

1	Project Profile	1
1.1	Introduction	1
1.2	Objective.....	1
1.3	Scope	2
2	Requirement Analysis	4
2.1	Target Audience	4
2.2	Hardware and Software	4
3	Website Planning and Structure.....	6
3.1	Sitemap.....	6
3.2	Key Pages	6
3.3	Timeline and Milestones	8
4	Data Dictionary	10
4.1	Tables	10
5	Diagrams	12
5.1	DFD Diagram	12
5.1.1	Context Level (Level-0).....	12
5.1.2	Level-1 Diagram	13
5.2	ER Diagram	15
5.3	Use Case Diagram	16
6	User Interface.....	18
6.1	Admin Panel	18
6.1.1	Login Page	18
6.1.2	Dashboard	21
6.1.3	Manage Employees Page	26
6.1.4	Manage Supplier Page.....	32
6.1.5	Manage Categories Page	36
6.1.6	Manage Products Page	39
6.1.7	Manage Sales	45
6.2	Employee Side.....	49
6.2.1	Login Page	49
6.2.2	Employee Dashboard	51
7	Budget and Financial Plan	63
7.1	Cost Estimation	63
7.2	Financial Planning.....	64
8	Future Enhancement	65

9	Conclusion	66
10	References.....	67

1 Project Profile

1.1 Introduction

The **Electronics Shop Inventory Management System** is a desktop-based application developed using Python's Tkinter library for the graphical user interface (GUI) and MySQL for the backend database. This system is designed to help electronics shops efficiently manage their inventory, ensuring accurate stock tracking and streamlined operations.

The primary objective of this project is to automate and simplify inventory-related tasks, such as managing product categories, tracking supplier information, monitoring stock levels, and maintaining employee details. By replacing traditional manual methods with this digital solution, the system minimizes errors, enhances productivity, and provides real-time access to critical data.

With an intuitive user interface and robust database integration, the Electronics Shop Inventory Management System ensures smooth functionality, making it a reliable tool for shop owners to manage their day-to-day business activities effectively.

1.2 Objective

The main objective of the **Electronics Shop Inventory Management System** is to provide an efficient and user-friendly solution for managing inventory operations in electronics shops. This system is designed to address the challenges of traditional manual inventory management by offering a streamlined and automated approach. The key objectives include:

Efficient Inventory Tracking

To accurately monitor the stock levels of electronics products, ensuring timely restocking and minimizing overstocking or stockouts.

Category Management

To organize products into categories for easy retrieval and better classification of inventory items.

Supplier Management

To maintain a comprehensive database of suppliers, including their details and supplied products, enabling seamless coordination and procurement.

Employee Management

To manage employee details, roles, and responsibilities, ensuring accountability and proper record-keeping.

Data Accuracy and Integrity

To minimize errors in inventory records by leveraging a centralized database, ensuring the integrity of critical data.

Tax Calculation and Management

To automate the calculation of applicable taxes, ensuring compliance and simplifying financial operations.

User-Friendly Interface

To provide an intuitive GUI for shop owners and employees, enabling easy navigation and efficient operation.

Scalability and Customization

To allow flexibility for adding more features or scaling the system as the shop grows.

1.3 Scope

The **Electronics Shop Inventory Management System** is a comprehensive solution designed to automate and streamline the management of inventory and related business operations in electronics shops. The system is tailored for small to medium-sized businesses and includes the following features:

1. Core Functionalities

Employee Management:

Add, update, and manage employee records, including roles and responsibilities.

Maintain employee activity logs to ensure accountability.

Supplier Management:

Maintain detailed supplier records, including contact information and products supplied.

Facilitate seamless coordination with suppliers for procurement.

Category Management:

Organize inventory into categories for efficient browsing and management.

Simplify the retrieval of products within specific categories.

Product Management:

Add, update, and delete product details, including pricing, stock levels, and descriptions.

Track product movement to monitor stock availability.

Billing System:

Generate detailed bills with tax calculations.

Maintain billing records for reporting and audit purposes.

2. Database Management

Centralized Data Storage:

Store all data securely using a MySQL database.

Enable quick retrieval and modification of records.

3. Tax and Financial Management

Automated calculation of applicable taxes.

Integration of tax details into billing for accurate financial reporting.

4. User Interface

GUI Integration:

A user-friendly interface built with Python Tkinter for easy navigation.

Simplified workflows for employees with minimal technical knowledge.

5. Reporting and Analysis

Generate reports for employee records, supplier activities, category details, stock levels, and billing data.

Use analytics to make informed decisions regarding inventory and business operations.

2 Requirement Analysis

2.1 Target Audience

The **Electronics Shop Inventory Management System** is designed to cater to the following primary audiences:

Electronics Shop Owners:

Business owners managing small to medium-sized electronics shops.

Individuals seeking a reliable and automated solution to handle inventory, suppliers, employees, and billing operations.

Owners aiming to streamline business processes, reduce manual effort, and improve accuracy in daily operations.

Employees:

Staff members responsible for managing inventory, updating product records, and handling billing.

Employees involved in day-to-day operations who require an easy-to-use system to enhance productivity and ensure smooth workflows.

2.2 Hardware and Software

Hardware Requirements

Processor:

Minimum: Dual-core processor

Recommended: Intel i5 or higher

RAM:

Minimum: 4 GB

Recommended: 8 GB or higher

Storage:

Minimum: 250 GB HDD

Recommended: 512 GB SSD or higher

Display:

Resolution: 1366 x 768 or higher

Screen Size: 15.6 inches or larger for optimal GUI display

Input Devices:

Keyboard and Mouse

Additional Peripherals:

Printer (for generating bills and reports, optional)

Software Requirements**Operating System:**

Windows 10 or higher

Linux (Ubuntu 18.04 or higher)

Programming Environment:

Python 3.7 or higher

Tkinter library for GUI

Database Management System:

MySQL 8.0 or higher

Additional Tools and Libraries:

MySQL Connector for Python

Required Python libraries (e.g., pandas, openpyxl for reporting if applicable)

Development Environment:

Integrated Development Environment (IDE) like PyCharm, Visual Studio Code, or any preferred Python IDE

Other Software:

XAMPP or WAMP (optional, for database management during development)

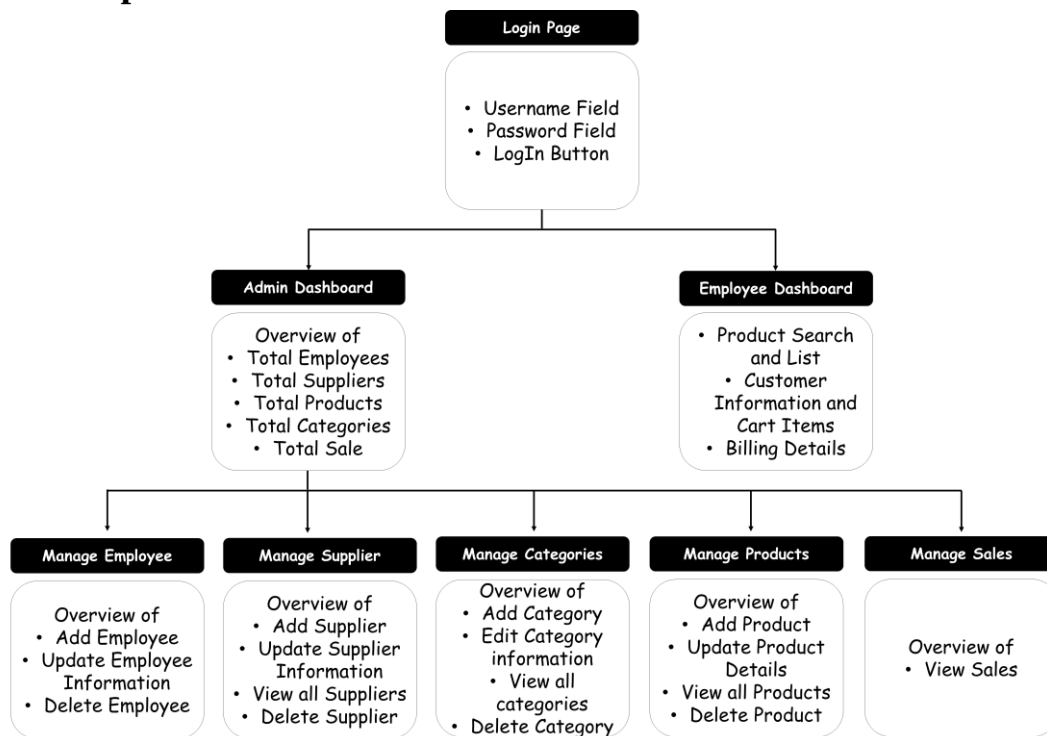
Drivers and Utilities:

Printer drivers (if using a printer)

Database backup utilities for MySQL

3 Website Planning and Structure

3.1 Sitemap



3.2 Key Pages

Dashboard

- **Purpose:**
 - Serves as the central hub of the system, providing a summary of key metrics and quick navigation to other pages.
- **Features:**
 - Displays overall statistics like total employees, suppliers, products, categories, and sales.
 - Includes shortcuts to critical functionalities such as employee, supplier, product, and category management.

Employee Management Page

- **Purpose:**
 - Allows the shop owner to manage employee details efficiently.
- **Features:**
 - Add, update, and delete employee records.
 - View employee details, including name, role, and contact information.
 - Manage employee roles and permissions.

Products Management Page

- **Purpose:**
 - Enables efficient management of product inventory.
- **Features:**
 - Add new products with details like name, price, stock quantity, and description.
 - Update or delete product records.
 - Search and filter products by category or stock level.
 - Monitor stock availability to avoid overstocking or stockouts.

Categories Management Page

- **Purpose:**
 - Facilitates the organization of products into categories for better management.
- **Features:**
 - Create new categories and assign products to them.
 - Update or delete existing categories.
 - View category-wise product counts for better organization.

Supplier Management Page

- **Purpose:**
 - Maintains records of suppliers for smooth procurement processes.
- **Features:**
 - Add, update, or delete supplier records.
 - View supplier details, including name, contact information, and supplied products.
 - Track supplier activity and manage relationships efficiently.

Sales Report Page

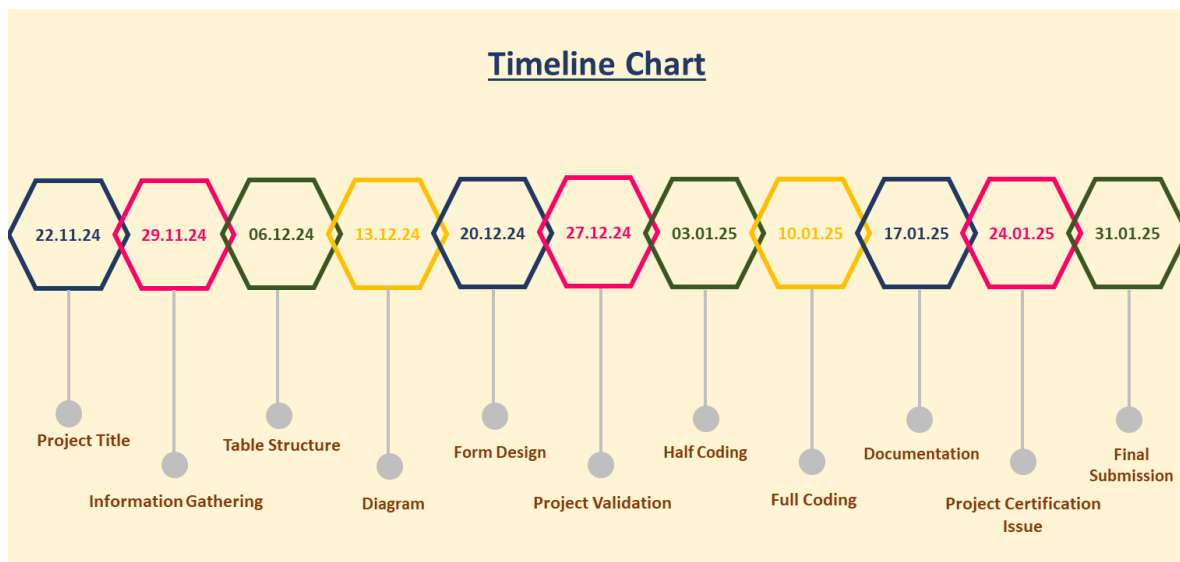
- **Purpose:**
 - Provides detailed insights into sales performance.
- **Features:**
 - Generate reports for daily, monthly, or custom date ranges.
 - Analyze sales trends and revenue statistics.
 - Export reports to formats like Excel or PDF for further analysis.

Billing Dashboard

- **Purpose:**
 - Streamlines the billing process for sales transactions.
- **Features:**
 - Generate invoices with itemized billing and tax details.
 - Apply discounts or promotional offers as needed.
 - Print or save invoices for record-keeping.
 - Maintain a log of all bills for future reference.

3.3 Timeline and Milestones

Timeline:



Milestones:

- **22.11.24 - Project Title & Information Gathering**
 - Finalize the project title: "Electronics Shop Inventory Management System."
 - Research the requirements and features needed for the system.
- **29.11.24 - Table Structure Design**
 - Design the database schema, including tables for employees, suppliers, products, categories, and billing.
- **06.12.24 - Diagram**
 - Create system architecture diagrams (ER diagram, flowchart, and use-case diagrams).
- **13.12.24 - Form Design**
 - Develop the GUI layouts for all key pages, such as the dashboard, employee management, and billing.
- **20.12.24 - Project Validation**
 - Ensure initial validation of forms and database connectivity for proper data handling.
- **27.12.24 - Half Coding Completion**
 - Complete 50% of the coding, including database integration and basic functionality for employee and product management.
- **03.01.25 - Full Coding Completion**
 - Finalize the coding for all modules, ensuring end-to-end functionality.
- **10.01.25 - Documentation**
 - Prepare comprehensive project documentation, including an introduction, objectives, scope, and key page descriptions.
- **17.01.25 - Project Certification Issue**
 - Receive certification from relevant authorities or instructors.
- **24.01.25 - Final Submission**
 - Submit the completed project, including code, documentation, and reports.

4 Data Dictionary

4.1 Tables

Table Name: categories

Field Name	Data Type	Field Length	Constraints	Description
category_id	INT	20	PRIMARY KEY	Product's Category Id, auto generated
category_name	VARCHAR	100	NOT NULL	Product's Category Name
description	TEXT	100	NOT NULL	Product's Category Description

Table Name: products

Field Name	Data Type	Field Length	Constraints	Description
product_id	INT	20	PRIMARY KEY	Product's Id, auto generated
product_name	VARCHAR	100	NOT NULL	Product's Name
category_id	INT	20	FOREIGN KEY	Product's Category id
price	DECIMAL	8,2	NOT NULL	Product's Price
quantity	INT	20	NOT NULL	Product's Quantity
supplier_id	INT	20	FOREIGN KEY	Product's Supplier id

Table Name: suppliers

Field Name	Data Type	Field Length	Constraints	Description
supplier_id	INT	20	PRIMARY KEY	Product's Supplier Id, auto generated
supplier_name	VARCHAR	100	NOT NULL	Product's Supplier Name
contact	VARCHAR	10	NOT NULL	Supplier's Contact Number
address	TEXT	100	NOT NULL	Supplier Address

Table Name: customers

Field Name	Data Type	Field Length	Constraints	Description
customer_id	INT	20	PRIMARY KEY	Customer's Id, auto generated
customer_name	VARCHAR	100	NOT NULL	Customer's Name
customer_contact	VARCHAR	10	NOT NULL	Customer's Contact Number
pur_products	TEXT	255	NOT NULL	Purchased Products
price	VARCHAR	10	NOT NULL	Price of Purchased Products
discount	VARCHAR	10	NOT NULL	Discount on Purchased Products
total_price	VARCHAR	10	NOT NULL	Total price

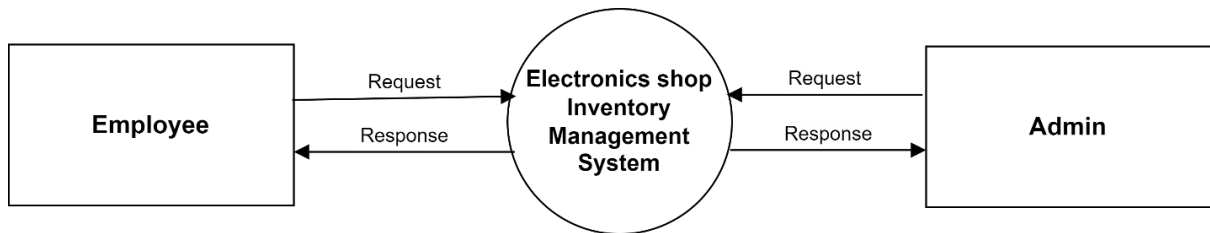
Table Name: employees

Field Name	Data Type	Field Length	Constraints	Description
employee_id	INT	20	PRIMARY KEY	Employee's Id, auto generated
employee_name	VARCHAR	100	NOT NULL	Employee's Name
employee_email	VARCHAR	100	NOT NULL	Employee's Email
employee_gender	VARCHAR	10	NOT NULL	Employee's Gender
employee_dob	VARCHAR	10	NOT NULL	Employee's Date Of Birth
employee_contact	VARCHAR	10	NOT NULL	Employee's Contact Number
employment_type	VARCHAR	100	NOT NULL	Employee's Employment type like Full Time, Part Time, Contract, Intern
employee_education	VARCHAR	100	NOT NULL	Employee's Education
employee_workshift	VARCHAR	100	NOT NULL	Employee's Work Shift like Morning, Evening and Night
employee_address	TEXT	100	NOT NULL	Employee's Address
employee_doj	VARCHAR	100	NOT NULL	Employee's Date Of Joining
employee_salary	VARCHAR	100	NOT NULL	Employee's Salary

5 Diagrams

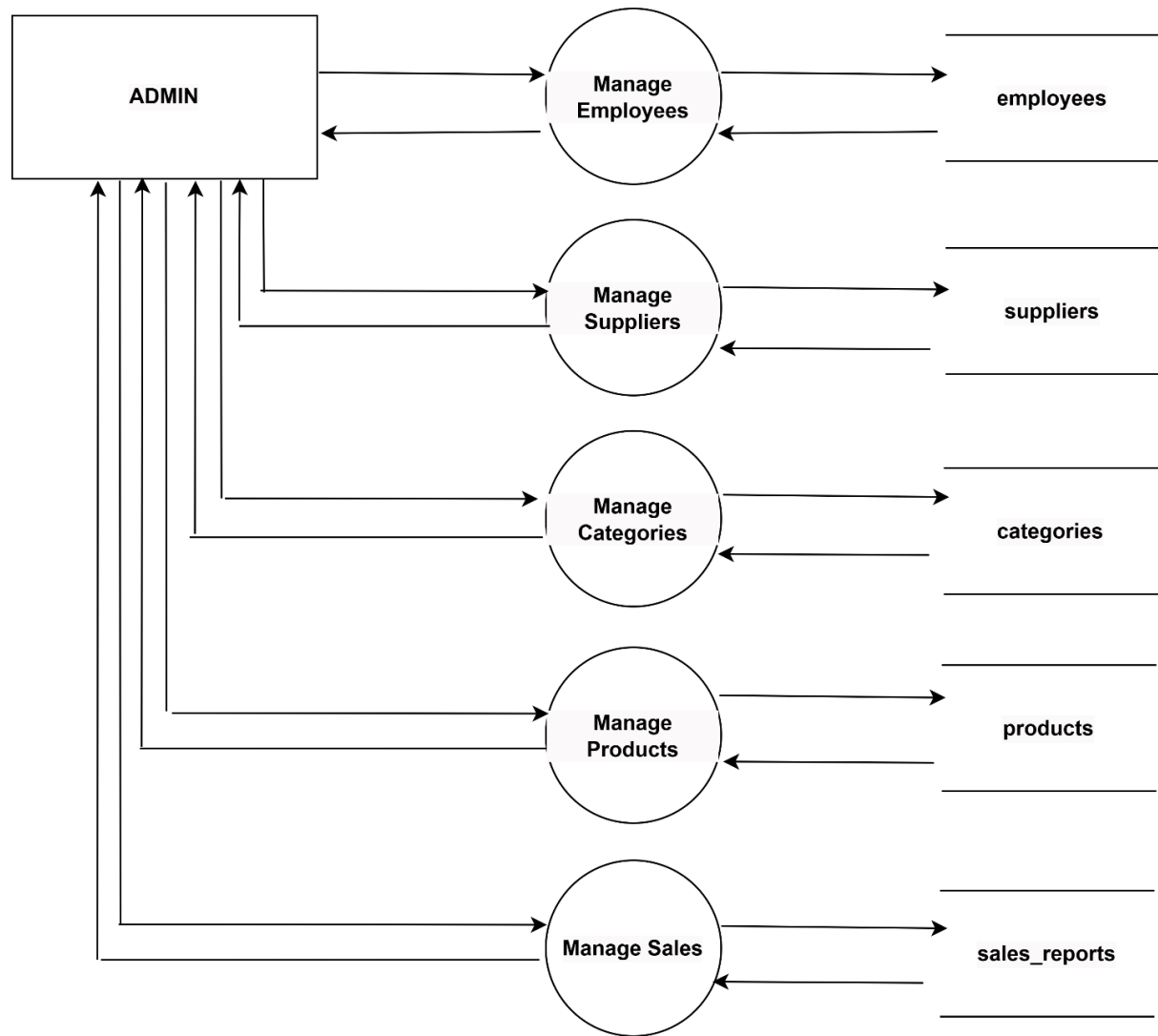
5.1 DFD Diagram

5.1.1 Context Level (Level-0)

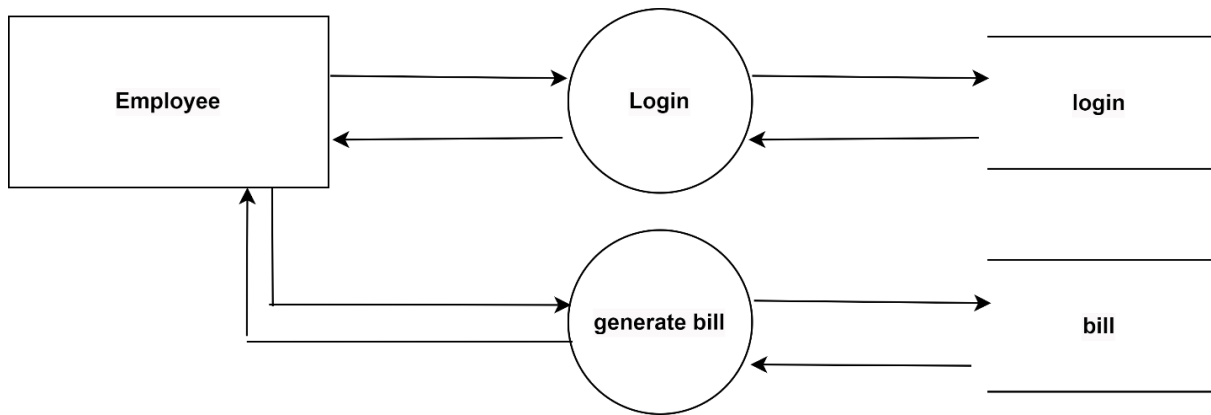


5.1.2 Level-1 Diagram

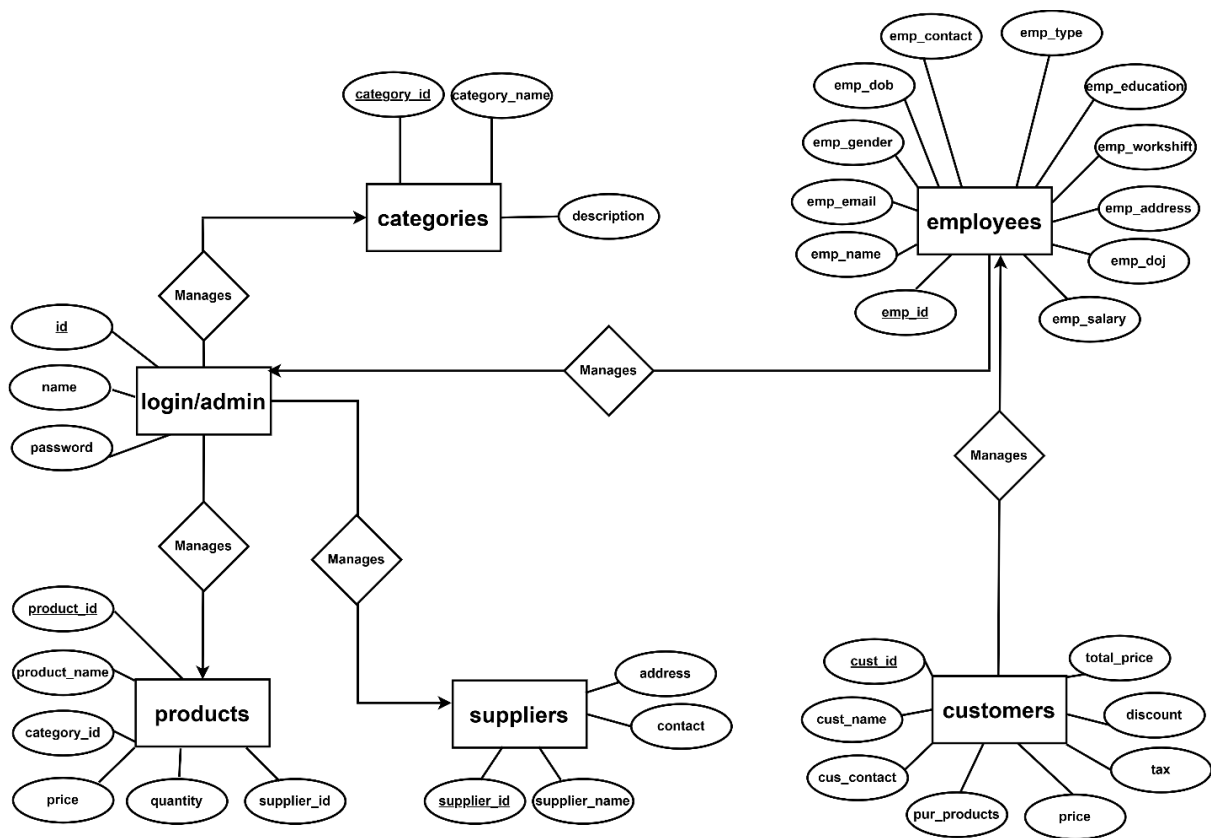
Admin Side



Employee

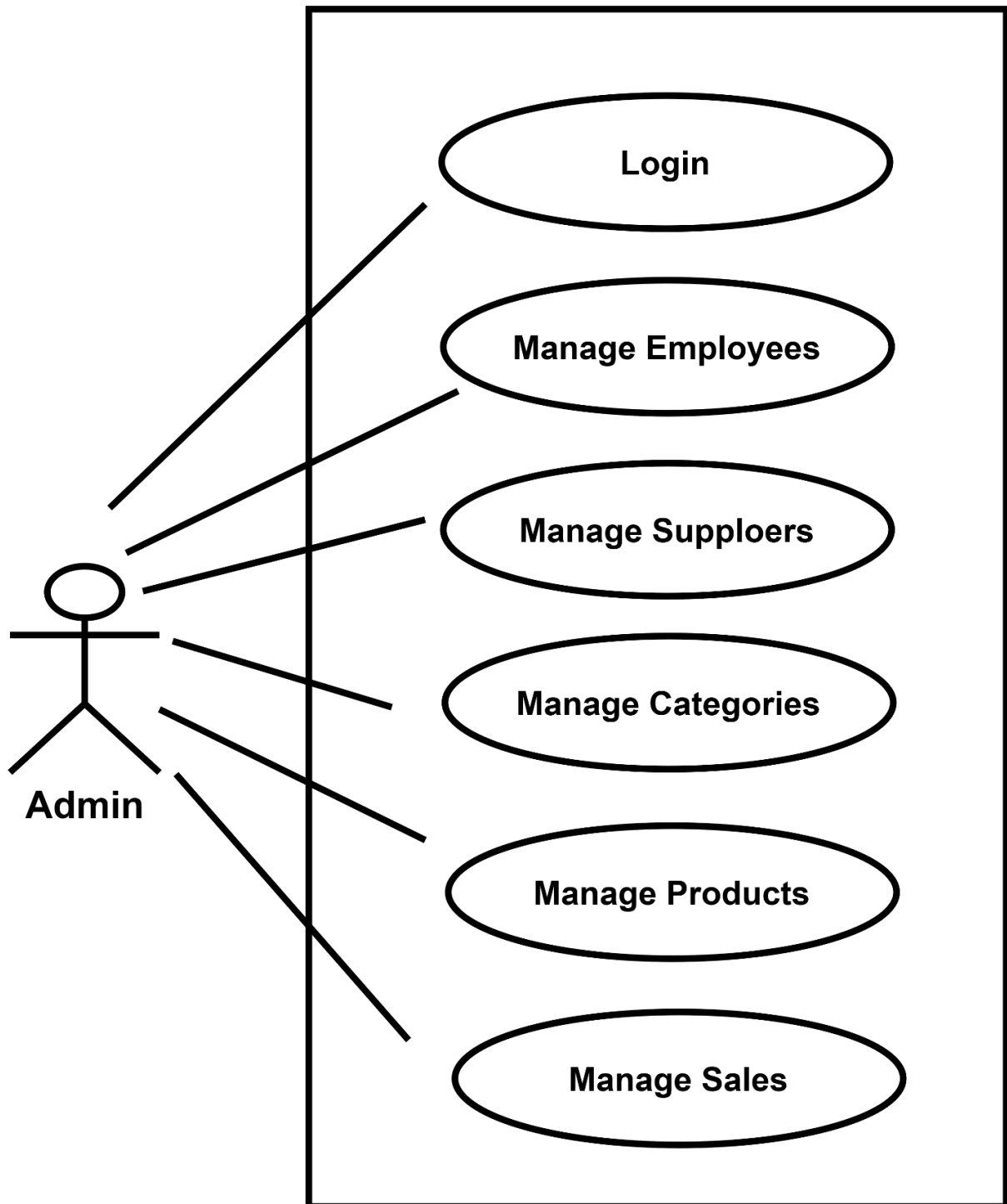


5.2 ER Diagram

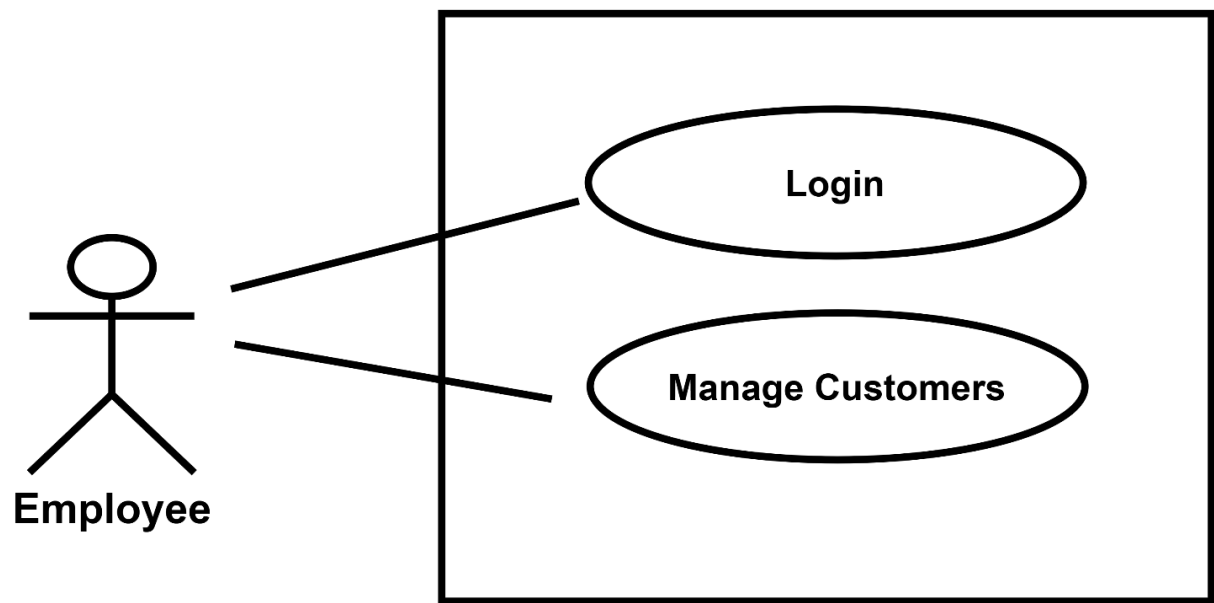


5.3 Use Case Diagram

Admin



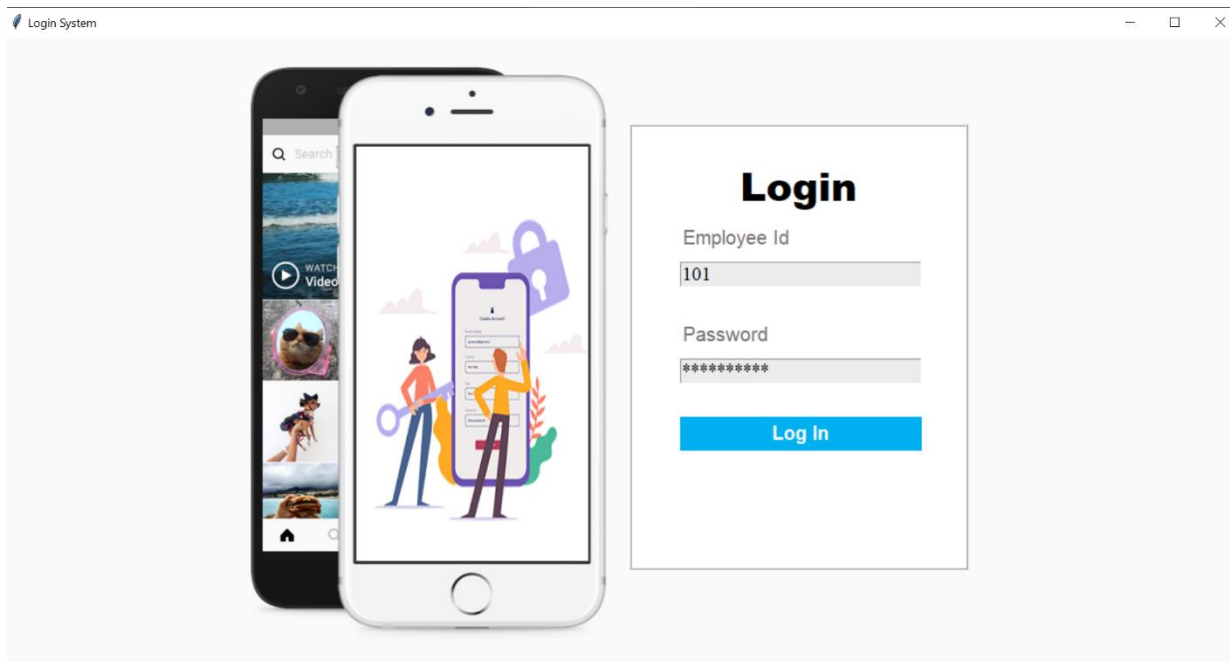
Employee



6 User Interface

6.1 Admin Panel

6.1.1 Login Page



Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import messagebox
import mysql.connector
import os

class Login_System:
    def __init__(self,root):
        self.root=root
        self.root.title("Login System")
        self.root.geometry("1350x700+0+0")
        self.root.config(bg="#fafafa")

        self.phone_image=ImageTk.PhotoImage(file="phone.png")
        self.lbl_Phone_image=Label(self.root,image=self.phone_image,bd=0).place(x=200,y=10)
```

```

self.employee_id=StringVar()
self.password=StringVar()

login_frame=Frame(self.root,bd=2,relief=RIDGE,bg="white")
login_frame.place(x=650,y=90,width=350,height=460)

title=Label(login_frame,text="Login", font=("Arial
Black",30,"bold"),bg="white").place(x=0,y=30,relwidth=1)

lbl_user=Label(login_frame,text="Employee
Id",font=("Andalus",15),bg="white",fg="#767171").place(x=50,y=100)
txt_employee_id=Entry(login_frame,textvariable=self.employee_id,font=(
'times new roman',15),bg="#ececce").place(x=50,y=140,width=250)

lbl_pass=Label(login_frame,text="Password",font=("Andalus",15),bg="whi
te",fg="#767171").place(x=50,y=200)
txt_pass=Entry(login_frame,textvariable=self.password,show="*",font=(
'times new roman',15),bg="#ececce").place(x=50,y=240,width=250)

btn_login=Button(login_frame,text="Log
In",command=self.login,font=("Arial Rounded MT
Bold",15,"bold"),bg="#00b0f0",fg="white",activebackground="#00b0f0",activefore
ground="white",bd=0,cursor="hand2").place(x=50,y=300,width=250,height=35)

"""hr=Label(login_frame,bg="lightgray").place(x=50,y=370,width=250,hei
ght=2)
or_=Label(login_frame,text="OR",bg="white",fg="lightgray",font=("times
new roman",15,"bold")).place(x=160,y=355)"""

#btn_forget=Button(login_frame,text="Forget
Password?",command=self.forget_window,font=("times new
roman",13),bg="white",fg="#00759e",activebackground="white",activeforeground="
#00759e",bd=0,cursor="hand2").place(x=110,y=390)

self.im1=ImageTk.PhotoImage(file="im1.png")
self.im2=ImageTk.PhotoImage(file="im2.png")
self.im3=ImageTk.PhotoImage(file="im3.png")

self.lbl_change_image=Label(self.root,bg="gray")
self.lbl_change_image.place(x=367,y=113,width=240,height=428)

self.animate()

def animate(self):
    self.im=self.im1
    self.im1=self.im2
    self.im2=self.im3

```

```
        self.im3=self.im
        self.lbl_change_image.config(image=self.im)
        self.lbl_change_image.after(1500,self.animate)
root=Tk()
obj=Login_System(root)
root.mainloop()
```

6.1.2 Dashboard



Code:

```
from tkinter import *
from PIL import ImageTk
import pymysql
from employees import employee_form
from supplier import supplier_form
from category import category_form
from products import product_form
from sales import sales_form

import os
from datetime import datetime # Import datetime module

# GUI Part
window = Tk()
window.title("Dashboard")
window.geometry("1270x668+0+0")
window.resizable(0, 0)
window.config(bg='white')

def redirect_to_login():
    window.destroy() # Close the current window
    os.system('python logine.py')
def exit_program():
    window.quit()
# Load background image
```



```

bg_image = PhotoImage(file='inventory.png')

# Title label with image and text
titleLabel = Label(window, image=bg_image, compound=LEFT, text="Inventory
Management System",
                    font=("times new roman", 40, "bold"), bg="#010c48",
fg="white", anchor="w", padx=20)
titleLabel.place(x=0, y=0, relwidth=1)

logoutButton = Button(window, text="Logout", font=('times new roman', 20,
'bold'), fg='#010c48', command=redirect_to_login, cursor="hand2")
logoutButton.place(x=1100, y=10)

subtitleLabel = Label(window, text="Welcome Admin\t\t Date:15-11-2024\t\t
Time:9:00:00pm", font=('times new roman', 15), bg='#4d636d', fg='white')
subtitleLabel.place(x=0, y=70, relwidth=1)

# Function to update date and time dynamically

# Call update_date_time() to start updating the label
update_date_time()

leftFrame = Frame(window)
leftFrame.place(x=0, y=102, width=200, height=555)

logoImage = PhotoImage(file='logo.png')
imageLabel = Label(leftFrame, image=logoImage)
imageLabel.grid(row=0, column=0)

menuLabel = Label(leftFrame, text="Menu", font=('times new roman', 20),
bg='#009688')
menuLabel.grid(row=1, column=0, pady=10)
menuLabel.place(width=200, y=150)

import os

# Button for Employees
dash_icon = ImageTk.PhotoImage(file='dash.png')
dash_button = Button(leftFrame, image=dash_icon, compound=LEFT, text="
Dashboard", font=('times new roman', 20, 'bold'), anchor="w", command=lambda:
os.system('python dashboard.py'))
dash_button.grid(row=2, column=0, pady=10, padx=10)
dash_button.place(width=200, y=185, height=50)

employee_icon = ImageTk.PhotoImage(file='employee.png')
employee_button = Button(leftFrame, image=employee_icon, compound=LEFT, text="
Employees", font=('times new roman', 20, 'bold'), anchor="w", command=lambda:
employee_form(window))

```

```

employee_button.grid(row=3, column=0, pady=10, padx=10)
employee_button.place(width=200, y=234, height=50)

# Remaining code for buttons, frames, and labels
# ...

supplier_icon=ImageTk.PhotoImage(file='supplier.png')
supplier_button=Button(leftFrame,image=supplier_icon,compound=LEFT,text="
Supplier",font=('times new roman',20,'bold'),anchor="w",command=lambda
:supplier_form(window))
supplier_button.grid(row=3,column=0,pady=10, padx=10)
supplier_button.place(width=200,y=284,height=50)

category_icon=ImageTk.PhotoImage(file='category.png')
category_button=Button(leftFrame,image=category_icon,compound=LEFT,text="
Categories",font=('times new
roman',20,'bold'),anchor="w",command=lambda:category_form(window))
category_button.grid(row=5,column=0,pady=10, padx=10)
category_button.place(width=200,y=334,height=50)

products_icon=ImageTk.PhotoImage(file='product.png')
products_button=Button(leftFrame,image=products_icon,compound=LEFT,text="
Products",font=('times new
roman',20,'bold'),anchor="w",command=lambda:product_form(window))
products_button.grid(row=6,column=0,pady=10, padx=10)
products_button.place(width=200,y=384,height=50)

sales_icon=ImageTk.PhotoImage(file='sales.png')
sales_button=Button(leftFrame,image=sales_icon,compound=LEFT,text="
Sales",font=('times new
roman',20,'bold'),anchor="w",command=lambda:sales_form(window))
sales_button.grid(row=7,column=0,pady=10, padx=10)
sales_button.place(width=200,y=434,height=50)

exit_icon = ImageTk.PhotoImage(file='exit.png')
exit_button = Button(leftFrame, image=exit_icon, compound=LEFT, text=" Exit",
font=('times new roman', 20, 'bold'), anchor="w", command=exit_program)
exit_button.grid(row=8, column=0, pady=10, padx=10)
exit_button.place(width=200, y=484, height=50)
try:
    conn = pymysql.connect(host='localhost', user='root', password='',
database='inventory_system')
    cursor = conn.cursor()

    # Execute queries to get counts
    cursor.execute("SELECT COUNT(*) FROM employee_data")
    total_employees = cursor.fetchone()[0]

```

```

cursor.execute("SELECT COUNT(*) FROM supplier_data")
total_suppliers = cursor.fetchone()[0]

cursor.execute("SELECT COUNT(*) FROM category_data")
total_categories = cursor.fetchone()[0]

cursor.execute("SELECT COUNT(*) FROM product_data")
total_products = cursor.fetchone()[0]

cursor.execute("SELECT COUNT(*) FROM bills") # Assuming you have a
'sales' table
total_sales = cursor.fetchone()[0]

finally:
    if conn:
        conn.close()

emp_frame = Frame(window, bg='#2c3e50', bd=3, relief='ridge')
emp_frame.place(x=400, y=100, height=170, width=280)
total_emp_icon = PhotoImage(file='staff.png')
total_emp_icon_label = Label(emp_frame, image=total_emp_icon, bg='#2c3e50')
total_emp_icon_label.grid(row=0, column=0, padx=0)
total_emp_icon_label.place(x=110)
total_emp_label = Label(emp_frame, text="Total Employees", bg='#2c3e50',
fg='white', font=('times new roman', 15, 'bold'))
total_emp_label.grid(row=1, column=0, padx=105, pady=10)
total_emp_label.place(x=70, y=50)
total_emp_count_label = Label(emp_frame, text=str(total_employees),
bg='#2c3e50', fg='white', font=('times new roman', 30, 'bold'))
total_emp_count_label.grid(row=2, column=0, padx=105, pady=10)
total_emp_count_label.place(x=130, y=80)

sup_frame=Frame(window,bg='#8e44ad',bd=3,relief='ridge')
sup_frame.place(x=800,y=100,height=170,width=280)
total_sup_icon=PhotoImage(file='suppliers.png')
total_sup_icon_label=Label(sup_frame,image=total_sup_icon,bg='#8e44ad')
total_sup_icon_label.grid(row=0,column=0,padx=0)
total_sup_icon_label.place(x=110)
total_sup_label=Label(sup_frame,text="Total
Suppliers",bg='#8e44ad',fg='white',font=('times new roman',15,'bold'))
total_sup_label.grid(row=1,column=0,padx=105, pady=10)
total_sup_label.place(x=70,y=50)
total_sup_count_label=Label(sup_frame,text=str(total_suppliers),bg='#8e44ad',f
g='white',font=('times new roman',30,'bold'))
total_sup_count_label.grid(row=2,column=0,padx=105, pady=10)
total_sup_count_label.place(x=130,y=80)

cat_frame=Frame(window,bg='#27ae60',bd=3,relief='ridge')
cat_frame.place(x=400,y=285,height=170,width=280)

```

```

total_cat_icon=PhotoImage(file='options.png')
total_cat_icon_label=Label(cat_frame,image=total_cat_icon,bg='#27ae60')
total_cat_icon_label.grid(row=0,column=0,padx=0,pady=30)
total_cat_icon_label.place(x=110,y=10)
total_cat_label=Label(cat_frame,text="Total
Categories",bg='#27ae60',fg='white',font=('times new roman',15,'bold'))
total_cat_label.grid(row=1,column=0,padx=105, pady=10)
total_cat_label.place(x=70,y=60)
total_cat_count_label=Label(cat_frame,text=str(total_categories),bg='#27ae60',
fg='white',font=('times new roman',30,'bold'))
total_cat_count_label.grid(row=2,column=0,padx=105, pady=10)
total_cat_count_label.place(x=130,y=90)

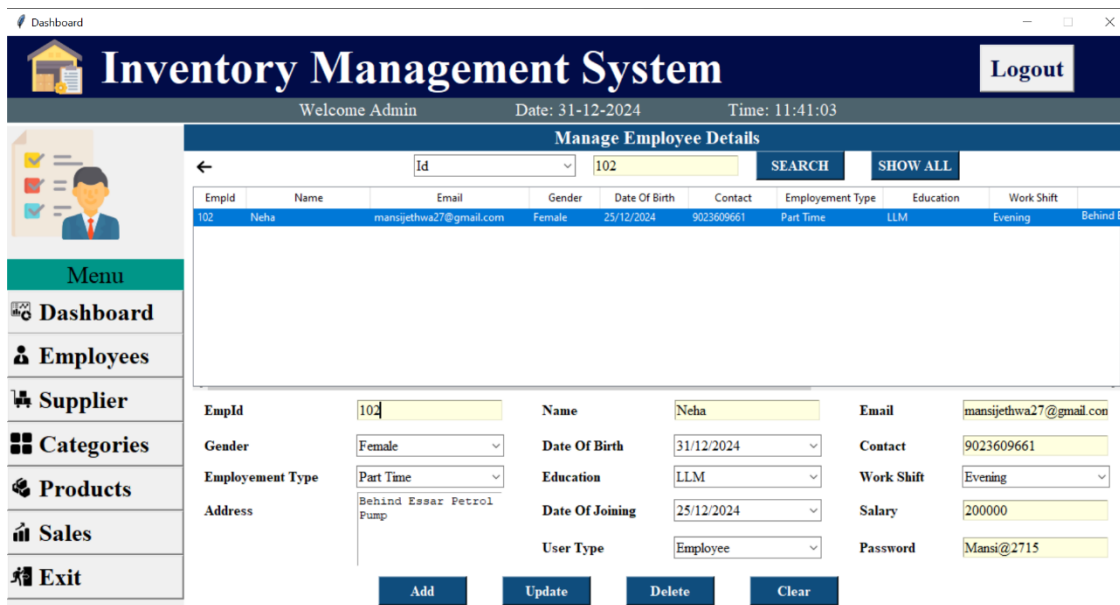
prod_frame=Frame(window,bg='#daa520',bd=3,relief='ridge')
prod_frame.place(x=800,y=285,height=170,width=280)
total_prod_icon=PhotoImage(file='products.png')
total_prod_icon_label=Label(prod_frame,image=total_prod_icon,bg='#daa520')
total_prod_icon_label.grid(row=0,column=0,padx=0)
total_prod_icon_label.place(x=110,y=10)
total_prod_label=Label(prod_frame,text="Total
Products",bg='#daa520',fg='white',font=('times new roman',15,'bold'))
total_prod_label.grid(row=1,column=0,padx=105, pady=10)
total_prod_label.place(x=70,y=65)
total_prod_count_label=Label(prod_frame,text=str(total_products),bg='#daa520',
fg='white',font=('times new roman',30,'bold'))
total_prod_count_label.grid(row=2,column=0,padx=105, pady=10)
total_prod_count_label.place(x=130,y=95)

sales_frame=Frame(window,bg='#de3163',bd=3,relief='ridge')
sales_frame.place(x=600,y=470,height=170,width=280)
total_sales_icon=PhotoImage(file='trend.png')
total_sales_icon_label=Label(sales_frame,image=total_sales_icon,bg='#de3163')
total_sales_icon_label.grid(row=0,column=0,padx=0)
total_sales_icon_label.place(x=110,y=10)
total_sales_label=Label(sales_frame,text="Total
Sales",bg='#de3163',fg='white',font=('times new roman',15,'bold'))
total_sales_label.grid(row=1,column=0,padx=105, pady=10)
total_sales_label.place(x=90,y=65)
total_sales_count_label=Label(sales_frame,text=str(total_sales),bg='#de3163',f
g='white',font=('times new roman',30,'bold'))
total_sales_count_label.grid(row=2,column=0,padx=105, pady=10)
total_sales_count_label.place(x=130,y=95)

# Main event loop
window.mainloop()

```

6.1.3 Manage Employees Page



Inventory Management System | Logout

Welcome Admin | Date: 31-12-2024 | Time: 11:41:03

Manage Employee Details

← Id: 102 [SEARCH] [SHOW ALL]

EmpId	Name	Email	Gender	Date Of Birth	Contact	Employment Type	Education	Work Shift	Behind
102	Neha	mansijethwa27@gmail.com	Female	25/12/2024	9023609661	Part Time	LLM	Evening	Behind

EmpId: 102 | **Name:** Neha | **Email:** mansijethwa27@gmail.com
Gender: Female | **Date Of Birth:** 31/12/2024 | **Contact:** 9023609661
Employment Type: Part Time | **Education:** LLM | **Work Shift:** Evening
Address: Behind Essar Petrol Pump | **Date Of Joining:** 25/12/2024 | **Salary:** 200000
User Type: Employee | **Password:** Mansi@2715

[Add] [Update] [Delete] [Clear]

Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry
import mysql.connector
from datetime import date
import re

import re

def employee_form(window):
    global back_image, employee_treeview
    employee_frame = Frame(window, width=1070, height=567, bg='white')
    employee_frame.place(x=200, y=100)
    heading_label = Label(employee_frame, text='Manage Employee
Details', font=('times new roman', 16, 'bold'), bg='#0f4d7d', fg='white')
    heading_label.place(x=0, y=0, relwidth=1)
    top_frame = Frame(employee_frame, bg='white')
    top_frame.place(x=0, y=30, relwidth=1)
    back_image = PhotoImage(file='arrow.png')
    back_button = Button(top_frame, image=back_image, bd=0,
cursor='hand2', bg='white', command=lambda: employee_frame.place_forget())
    back_button.place(x=10, y=5)
    search_frame = Frame(top_frame, bg='white')
    search_frame.place(x=240)
```

```

        search_combobox=ttk.Combobox(search_frame,values=('Id','Name','Email'),font=
t=('times new roman',12),state='readonly')
        search_combobox.set('Search By')
        search_combobox.grid(row=0,column=0,padx=20)
        search_entry=Entry(search_frame,font=('times new
roman',12),bg='lightyellow')
        search_entry.grid(row=0,column=1)
        search_button=Button(search_frame,text='SEARCH',width=10,font=('times new
roman',12,'bold'),bg="#0f4d7d", fg="white", cursor='hand2',command=lambda
:search_employee(search_combobox.get(),search_entry.get()))
        search_button.grid(row=0,column=2,padx=20)
        show_button=Button(search_frame,text='SHOW ALL',width=10,font=('times new
roman',12,'bold'),bg="#0f4d7d", fg="white", cursor='hand2',command=lambda
:show_all(search_entry,search_combobox))
        show_button.grid(row=0,column=3,padx=10)

        # Add the Treeview and scrollbars
        horizontal_scrollbar = Scrollbar(top_frame, orient=HORIZONTAL)
        vertical_scrollbar = Scrollbar(top_frame, orient=VERTICAL)

# Create the Treeview and link scrollbars
        employee_treeview = ttk.Treeview(
            top_frame,
            columns=(
                'empid', 'name', 'email', 'gender', 'dob', 'contact',
                'employment_type', 'education', 'work_shift', 'address',
                'doj', 'salary', 'usertype'
            ),
            show='headings',
            yscrollcommand=vertical_scrollbar.set,
            xscrollcommand=horizontal_scrollbar.set
        )

# Configure scrollbars to work with the Treeview
        horizontal_scrollbar.config(command=employee_treeview.xview)
        vertical_scrollbar.config(command=employee_treeview.yview)

# Place Treeview and scrollbars
        employee_treeview.place(x=100,y=200,height=200,width=100)
        employee_treeview.grid(row=0, column=0, padx=10, pady=40)
        vertical_scrollbar.grid(row=0, column=1,sticky='ns')
        horizontal_scrollbar.grid(row=1, column=0, sticky='ew', padx=10)

        horizontal_scrollbar.place(x=10,y=255,width=1050)
        vertical_scrollbar.place(x=1050,y=50,height=200)

# Define Treeview columns and headings

```

```

employee_treeview.heading('empid', text='EmpId')
employee_treeview.heading('name', text='Name')
employee_treeview.heading('email', text='Email')
employee_treeview.heading('gender', text='Gender')
employee_treeview.heading('dob', text='Date Of Birth')
employee_treeview.heading('contact', text='Contact')
employee_treeview.heading('employment_type', text='Employement Type')
employee_treeview.heading('education', text='Education')
employee_treeview.heading('work_shift', text='Work Shift')
employee_treeview.heading('address', text='Address')
employee_treeview.heading('doj', text='Date of Joining')
employee_treeview.heading('salary', text='Salary')
employee_treeview.heading('usertype', text='User Type')

employee_treeview.column('empid', width=60)
employee_treeview.column('name', width=140)
employee_treeview.column('email', width=180)
employee_treeview.column('gender', width=80)
employee_treeview.column('dob', width=100)
employee_treeview.column('contact', width=100)
employee_treeview.column('employment_type', width=120)
employee_treeview.column('education', width=120)
employee_treeview.column('work_shift', width=100)
employee_treeview.column('address', width=200)
employee_treeview.column('doj', width=100)
employee_treeview.column('salary', width=140)
employee_treeview.column('usertype', width=120)

treeview_data()

top_frame.grid_rowconfigure(0, weight=1)
top_frame.grid_columnconfigure(0, weight=1)

detail_frame=Frame(employee_frame,bg='white')
detail_frame.place(x=0,y=300)

empid_label=Label(detail_frame,text='EmpId',font=('times new
roman',12,'bold'),bg='white')
empid_label.grid(row=0,column=0,padx=20,pady=10,sticky="w")
empid_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')
empid_entry.grid(row=0,column=1,padx=20,pady=10)

name_label=Label(detail_frame,text='Name',font=('times new
roman',12,'bold'),bg='white')
name_label.grid(row=0,column=2,padx=20,pady=10,sticky="w")
name_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')

```



```

name_entry.grid(row=0,column=3,padx=20,pady=10)

email_label=Label(detail_frame,text='Email',font=('times new
roman',12,'bold'),bg='white')
email_label.grid(row=0,column=4,padx=20,pady=10,sticky="w")
email_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')
email_entry.grid(row=0,column=5,padx=20,pady=10)

gender_label=Label(detail_frame,text='Gender',font=('times new
roman',12,'bold'),bg='white')
gender_label.grid(row=1,column=0,padx=20,pady=5,sticky="w")
gender_combobox=ttk.Combobox(detail_frame,values=('Male','Female'),font=('
times new roman',12),width=18,state='readonly')
gender_combobox.set('Select Gender')
gender_combobox.grid(row=1,column=1,padx=20,pady=5)

dob_label=Label(detail_frame,text='Date Of Birth',font=('times new
roman',12,'bold'),bg='white')
dob_label.grid(row=1,column=2,padx=20,pady=5,sticky="w")
dob_date_entry=DateEntry(detail_frame,width=18,font=('times new
roman',12),state='readonly',date_pattern='dd/mm/yyyy')
dob_date_entry.grid(row=1,column=3,padx=20,pady=5)

contact_label=Label(detail_frame,text='Contact',font=('times new
roman',12,'bold'),bg='white')
contact_label.grid(row=1,column=4,padx=20,pady=5,sticky="w")
contact_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')
contact_entry.grid(row=1,column=5,padx=20,pady=5)

employment_type_label=Label(detail_frame,text='Employement
Type',font=('times new roman',12,'bold'),bg='white')
employment_type_label.grid(row=2,column=0,padx=20,pady=5,sticky="w")
employment_type_combobox=ttk.Combobox(detail_frame,values=('Full
Time','Part Time','Casual','Contract','Intern'),font=('times new
roman',12),width=18,state='readonly')
employment_type_combobox.set('Select Employment Type')
employment_type_combobox.grid(row=2,column=1,padx=20,pady=5)

education_label=Label(detail_frame,text='Education',font=('times new
roman',12,'bold'),bg='white')
education_label.grid(row=2,column=2,padx=20,pady=5,sticky="w")
education_options=['B.Tech','B.Com','M.Tech','B.Sc','M.Sc','BBA','MBA','LL
B','LLM','B.Arch','M.Arch']
education_combobox=ttk.Combobox(detail_frame,values=education_options,font
=('times new roman',12),width=18,state='readonly')
education_combobox.set('Select Education')

```



```

education_combobox.grid(row=2,column=3,padx=20,pady=5)

work_shift_label=Label(detail_frame,text='Work Shift',font=('times new
roman',12,'bold'),bg='white')
work_shift_label.grid(row=2,column=4,padx=20,pady=5,sticky="w")
work_shift_combobox=ttk.Combobox(detail_frame,values=('Morning','Evening',
'Night'),font=('times new roman',12),width=18,state='readonly')
work_shift_combobox.set('Select Work Shift')
work_shift_combobox.grid(row=2,column=5,padx=20,pady=5)

address_label=Label(detail_frame,text='Address',font=('times new
roman',12,'bold'),bg='white')
address_label.grid(row=3,column=0,padx=20,pady=5,sticky='w')
address_text=Text(detail_frame,width=20,height=5)
address_text.grid(row=3,column=1,rowspan=2)

doj_label=Label(detail_frame,text='Date Of Joining',font=('times new
roman',12,'bold'),bg='white')
doj_label.grid(row=3,column=2,padx=20,pady=5,sticky="w")
doj_date_entry=DateEntry(detail_frame,width=18,font=('times new
roman',12),state='readonly',date_pattern='dd/mm/yyyy')
doj_date_entry.grid(row=3,column=3,padx=20,pady=5)

usertype_label=Label(detail_frame,text='User Type',font=('times new
roman',12,'bold'),bg='white')
usertype_label.grid(row=4,column=2,padx=20,pady=5,sticky="w")
usertype_combobox=ttk.Combobox(detail_frame,values=('Admin','Employee'),fo
nt=('times new roman',12),width=18,state='readonly')
usertype_combobox.set('Select User Type')
usertype_combobox.grid(row=4,column=3,padx=20,pady=5)

salary_label=Label(detail_frame,text='Salary',font=('times new
roman',12,'bold'),bg='white')
salary_label.grid(row=3,column=4,padx=20,pady=5,sticky="w")
salary_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')
salary_entry.grid(row=3,column=5,padx=20,pady=5)

password_label=Label(detail_frame,text='Password',font=('times new
roman',12,'bold'),bg='white')
password_label.grid(row=4,column=4,padx=20,pady=5,sticky="w")
password_entry=Entry(detail_frame,font=('times new
roman',12),bg='lightyellow')
password_entry.grid(row=4,column=5,padx=20,pady=5)

button_frame=Frame(employee_frame,bg='white')
button_frame.place(x=200,y=510)

```

```

        ad_button=Button(button_frame,text='Add',font=('times new
roman',12,'bold')
,width=10,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda
:add_employee(empid_entry.get(),name_entry.get(),email_entry.get(),gender_comb
obox.get(),dob_date_entry.get(),contact_entry.get(),employment_type_combobox.
get(),education_combobox.get(),work_shift_combobox.get(),address_text.get(1.0,
END),doj_date_entry.get(),salary_entry.get(),usertype_combobox.get(),password_
entry.get()))
        ad_button.grid(row=0,column=0,padx=20)

        update_button=Button(button_frame,text='Update',font=('times new
roman',12,'bold')
,width=10,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda
:update_employee(empid_entry.get(),name_entry.get(),email_entry.get(),gender_c
ombobox.get(),dob_date_entry.get(),contact_entry.get(),employment_type_combob
ox.get(),education_combobox.get(),work_shift_combobox.get(),address_text.get(1
.0,END),doj_date_entry.get(),salary_entry.get(),usertype_combobox.get(),passwo
rd_entry.get()))
        update_button.grid(row=0,column=1,padx=20)

        delete_button=Button(button_frame,text='Delete',font=('times new
roman',12,'bold')
,width=10,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda
:delete_employee(empid_entry.get(),))
        delete_button.grid(row=0,column=2,padx=20)

        clear_button=Button(button_frame,text='Clear',font=('times new
roman',12,'bold')
,width=10,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda
:clear_fields(empid_entry,name_entry,email_entry,dob_date_entry,gender_combobo
x,contact_entry,employment_type_combobox,education_combobox,work_shift_combob
ox,address_text,doj_date_entry,salary_entry,usertype_combobox,password_entry,T
rue))
        clear_button.grid(row=0,column=3,padx=20)
        employee_treeview.bind('<ButtonRelease-1>',lambda
event:select_data(event,empid_entry,name_entry,email_entry,dob_date_entry,gend
er_combobox,contact_entry,employment_type_combobox,education_combobox,work_sh
ift_combobox,address_text,doj_date_entry,salary_entry,usertype_combobox,passwo
rd_entry))

```

6.1.4 Manage Supplier Page

The screenshot shows the 'Manage Supplier Details' page of an 'Inventory Management System'. The page has a dark blue header with the system name and a 'Logout' button. Below the header, a status bar shows 'Welcome Admin', 'Date: 31-12-2024', and 'Time: 11:41:19'. A left sidebar contains a 'Menu' with options: Dashboard, Employees, Supplier, Categories, Products, Sales, and Exit. The main content area is titled 'Manage Supplier Details' and features a back arrow. It includes input fields for 'Invoice No:' (103), 'Supplier Name:' (Amit Jethwa), 'Supplier Contact:' (9023609662), and 'Description:' (Supplier of TV). Below these are 'Add', 'Update', 'Delete', and 'Clear' buttons. To the right, there is a search section with 'Invoice No:' (103), 'Search', and 'Show all' buttons. Below the search is a table with the following data:

Invoice Id	Supplier Name	Supplier Contact	
103	Amit Jethwa	9023609662	Supplier of TV

Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry
import mysql.connector
from datetime import date

def validate_inputs(invoice, name, contact, description):
    """
    Validates the inputs for the supplier form.
    """
    if not invoice.isnumeric():
        messagebox.showerror("Error", "Invoice must be a numeric value.")
        return False
    if not name.strip():
        messagebox.showerror("Error", "Supplier name is required.")
        return False
    if not all(x.isalpha() or x.isspace() for x in name):
        messagebox.showerror("Error", "Supplier name must contain only
alphabetic characters and spaces.")
        return False
    if not contact.strip():
        messagebox.showerror("Error", "Supplier contact is required.")
        return False
    if not contact.isdigit() or len(contact) != 10:
```

```

        messagebox.showerror("Error", "Supplier contact must be a 10-digit
number.")
        return False
    if not description.strip():
        messagebox.showerror("Error", "Description is required.")
        return False
    return True
def supplier_form(window):
    global back_image
    supplier_frame=Frame(window,width=1070,height=567,bg='white')
    supplier_frame.place(x=200,y=100)

    heading_Label=Label(supplier_frame,text='Manage Supplier
Details',font=('times new roman',16,'bold'),bg='#0f4d7d',fg='white')
    heading_Label.place(x=0,y=0,relwidth=1)

    back_image=PhotoImage(file='arrow.png')
    back_button = Button(supplier_frame, image=back_image, bd=0,
cursor='hand2',bg='white', command=lambda: supplier_frame.place_forget())
    back_button.place(x=10,y=30)

    left_frame=Frame(supplier_frame,bg='white')
    left_frame.place(x=10,y=100)

    invoice_label=Label(left_frame,text='Invoice No:',font=('times new
roman',14,'bold'),bg='white')
    invoice_label.grid(row=0,column=0,padx=20,sticky="w")
    invoice_entry=Entry(left_frame,font=('times new
roman',14,'bold'),bg='lightyellow')
    invoice_entry.grid(row=0,column=1)

    name_label=Label(left_frame,text='Supplier Name:',font=('times new
roman',14,'bold'),bg='white')
    name_label.grid(row=1,column=0,padx=20,pady=20,sticky="w")
    name_entry=Entry(left_frame,font=('times new
roman',14,'bold'),bg='lightyellow')
    name_entry.grid(row=1,column=1)

    contact_label=Label(left_frame,text='Supplier Contact:',font=('times new
roman',14,'bold'),bg='white')
    contact_label.grid(row=2,column=0,padx=20,sticky="w")
    contact_entry=Entry(left_frame,font=('times new
roman',14,'bold'),bg='lightyellow')
    contact_entry.grid(row=2,column=1)

    description_label=Label(left_frame,text='Description:',font=('times new
roman',14,'bold'),bg='white')
    description_label.grid(row=3,column=0,padx=20,sticky="nw",pady=25)

```

```

description_text=Text(left_frame,width=25,height=6,bg='lightyellow')
description_text.grid(row=3,column=1,pady=20)

buttonFrame=Frame(left_frame,bg='white')
buttonFrame.grid(row=4,columnspan=2,pady=20)

# Assuming you have a frame or container for buttons
add_button = Button(buttonFrame, text='Add', font=('times new roman', 14),
width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                    command=lambda: add_supplier(invoice_entry.get(),
name_entry.get(), contact_entry.get(), description_text.get(1.0, END).strip(),
treeview,invoice_entry, name_entry, contact_entry, description_text))
add_button.grid(row=0, column=0, padx=20)

# Create the Update button and bind it to the update_supplier function
update_button = Button(buttonFrame, text='Update', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                    command=lambda: update_supplier(
                        invoice_entry.get(),
                        name_entry.get(),
                        contact_entry.get(),
                        description_text.get("1.0", "end-1c"),
                        treeview,
                        invoice_entry,
                        name_entry,
                        contact_entry,
                        description_text
                    ))

update_button.grid(row=0, column=1)

delete_button = Button(buttonFrame, text='Delete', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                    command=lambda: delete_supplier(treeview,invoice_entry,
name_entry, contact_entry, description_text))
delete_button.grid(row=0, column=2, padx=20)

cancel_button = Button(buttonFrame, text='Clear', font=('times new roman',
14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                    command=lambda: clear(invoice_entry, name_entry,
contact_entry, description_text))
cancel_button.grid(row=0, column=3)

right_frame=Frame(supplier_frame,bg='white')
right_frame.place(x=520,y=80,width=500,height=350)

```

```

search_frame=Frame(right_frame,bg='white')
search_frame.pack(pady=10)

num_label=Label(search_frame,text='Invoice No:',font=('times new
roman',14,'bold'),bg='white')
num_label.grid(row=0,column=0,padx=15,sticky="w")
search_entry=Entry(search_frame,font=('times new
roman',14,'bold'),bg='lightyellow',width=10)
search_entry.grid(row=0,column=1)

search_button = Button(search_frame, text='Search', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                      command=lambda: search_supplier(search_entry,
treeview))
search_button.grid(row=0, column=2, padx=15)

show_button=Button(search_frame,text='Show all',font=('times new
roman',14) ,width=8,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda:
show_all(treeview,search_entry))
show_button.grid(row=0,column=3)

scrolly=Scrollbar(right_frame,orient=VERTICAL)
scrollx=Scrollbar(right_frame,orient=HORIZONTAL)
treeview=ttk.Treeview(right_frame,column=('invoice','name','contact','desc
ription'),show='headings',yscrollcommand=scrolly.set,xscrollcommand=scrollx.se
t)
scrolly.pack(side=RIGHT,fill=Y)
scrollx.pack(side=BOTTOM,fill=X)
scrollx.config(command=treeview.xview)
scrolly.config(command=treeview.yview)
treeview.pack(fill=BOTH,expand=1)
treeview.heading('invoice',text='Invoice Id')
treeview.heading('name',text='Supplier Name')
treeview.heading('contact',text='Supplier Contact')
treeview.heading('description',text='Description')

treeview.column('invoice',width=80)
treeview.column('name',width=160)
treeview.column('contact',width=120)
treeview.column('description',width=300)

treeview_data(treeview)
treeview.bind('<ButtonRelease-1>',lambda
event:select_data(event,invoice_entry,name_entry,contact_entry,description_tex
t,treeview))

```

6.1.5 Manage Categories Page

Dashboard

Welcome Admin Date: 31-12-2024 Time: 11:41:27

Inventory Management System Logout

Menu

- Dashboard
- Employees
- Supplier
- Categories
- Products
- Sales
- Exit

Manage Categories Details

Id: 103

Category Name: Refrigerator

Description: Fridge

Add Delete Clear

Id	Category Name	Description
101	TV	Tv
102	Phones	Phones
103	Refrigerator	Fridge

Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry
import mysql.connector
from datetime import date

# Function to connect to the database and set up the necessary table
# Function to connect to the database and set up the necessary table
def connect_database():
    mydb = mysql.connector.connect(host="localhost", user="root", password="")
    mycursor = mydb.cursor()
    mycursor.execute("CREATE DATABASE IF NOT EXISTS inventory_system")
    mycursor.execute("USE inventory_system")
    mycursor.execute("CREATE TABLE IF NOT EXISTS category_data (id INT PRIMARY KEY,name VARCHAR(100),description TEXT)")
    print("Database and Category Table are ready.")
    return mycursor
connect_database()
# Function to clear the input fields
# Function to clear the input fields
def clear(id_entry, category_name_entry, description_text):
    id_entry.delete(0, END)
    category_name_entry.delete(0, END)
    description_text.delete(1.0, END)
```

```

# Function to load data into the Treeview
def category_form(window):
    global back_image, logo

    # Frame for category management
    category_frame = Frame(window, width=1070, height=567, bg='white')
    category_frame.place(x=200, y=100)

    heading_Label = Label(category_frame, text='Manage Categories Details',
font=('times new roman', 16, 'bold'), bg='#0f4d7d', fg='white')
    heading_Label.place(x=0, y=0, relwidth=1)

    back_image = PhotoImage(file='arrow.png')
    back_button = Button(category_frame, image=back_image, bd=0,
cursor='hand2', bg='white', command=lambda: category_frame.place_forget())
    back_button.place(x=10, y=30)

    logo = PhotoImage(file='cat4.png')
    label = Label(category_frame, image=logo, bg="white")
    label.place(x=30, y=100)

    details_frame = Frame(category_frame, bg="white")
    details_frame.place(x=500, y=60)

    id_label = Label(details_frame, text='Id:', font=('times new roman', 14,
'bold'), bg='white')
    id_label.grid(row=0, column=0, padx=20, sticky="w")
    id_entry = Entry(details_frame, font=('times new roman', 14, 'bold'),
bg='lightyellow')
    id_entry.grid(row=0, column=1)

    category_name_label = Label(details_frame, text='Category Name:',
font=('times new roman', 14, 'bold'), bg='white')
    category_name_label.grid(row=1, column=0, padx=20, sticky="w")
    category_name_entry = Entry(details_frame, font=('times new roman', 14,
'bold'), bg='lightyellow')
    category_name_entry.grid(row=1, column=1, pady=20)

    description_label = Label(details_frame, text='Description:', font=('times
new roman', 14, 'bold'), bg='white')
    description_label.grid(row=2, column=0, padx=20, sticky="nw")
    description_text = Text(details_frame, width=25, height=6,
bg='lightyellow')
    description_text.grid(row=2, column=1)

    button_frame = Frame(category_frame, bg='white')
    button_frame.place(x=580, y=280)

```



```

        add_button = Button(button_frame, text='Add', font=('times new roman',
14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                           command=lambda:
add_category(id_entry.get(),category_name_entry.get(),
description_text.get(1.0, END), treeview, id_entry, category_name_entry,
description_text))
        add_button.grid(row=0, column=0, padx=20)

        delete_button = Button(button_frame, text='Delete', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                              command=lambda: delete_category(treeview))
        delete_button.grid(row=0, column=1, padx=20)

        clear_button = Button(button_frame, text='Clear', font=('times new roman',
14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                              command=lambda: clear(id_entry, category_name_entry,
description_text))
        clear_button.grid(row=0, column=2, padx=20)

        treeview_frame = Frame(category_frame, bg='white')
        treeview_frame.place(x=530, y=340, height=200, width=500)

        scrolly = Scrollbar(treeview_frame, orient=VERTICAL)
        scrollx = Scrollbar(treeview_frame, orient=HORIZONTAL)
        treeview = ttk.Treeview(treeview_frame, column=('id', 'name',
'description'), show='headings',
                               yscrollcommand=scrolly.set,
xscrollcommand=scrollx.set)
        scrolly.pack(side=RIGHT, fill=Y)
        scrollx.pack(side=BOTTOM, fill=X)
        scrollx.config(command=treeview.xview)
        scrolly.config(command=treeview.yview)
        treeview.pack(fill=BOTH, expand=1)

        treeview.heading('id', text='Id')
        treeview.heading('name', text='Category Name')
        treeview.heading('description', text='Description')

        treeview.column('id', width=80)
        treeview.column('name', width=160)
        treeview.column('description', width=300)

        treeview_data(treeview)
        treeview.bind('<ButtonRelease-1>', lambda event: select_data(event,
id_entry, category_name_entry, description_text, treeview))

```

6.1.6 Manage Products Page

The screenshot shows the 'Manage Products Details' page of an 'Inventory Management System'. The page has a dark blue header with the system name and a 'Logout' button. Below the header, a status bar shows 'Welcome Admin', 'Date: 31-12-2024', and 'Time: 11:41:53'. On the left is a sidebar menu with options: Dashboard, Employees, Supplier, Categories, Products, Sales, and Exit. The main content area is divided into two sections. The left section, titled 'Manage Products Details', contains form fields for Category (Refrigerator), Supplier (Kajal Jethwa), Product Name (LG Fridge), Price (20000.00), Quantity (21), and Status (Active). Below these fields are buttons for 'Add', 'Update', 'Delete', and 'Clear'. The right section, titled 'Search Product', has a search bar with 'Refrigerator' entered and buttons for 'Search' and 'Show all'. Below the search bar is a table with columns: Id, Category, Supplier, and Name. The table contains one row with the following data: Id: 2, Category: Refrigerator, Supplier: Kajal Jethwa, Name: LG Fridge.

Id	Category	Supplier	Name
2	Refrigerator	Kajal Jethwa	LG Fridge

Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry
import mysql.connector
from datetime import date

def validate_update_input(category, supplier, name, price, quantity, status):
    """Validates the input fields for product updates."""
    if category == 'Select':
        messagebox.showerror('Error', 'Please select a Category.')
        return False
    if supplier == 'Select':
        messagebox.showerror('Error', 'Please select a Supplier.')
        return False
    if not name:
        messagebox.showerror('Error', 'Please enter a Product Name.')
        return False
    try:
        float(price)
    except ValueError:
        messagebox.showerror('Error', 'Price must be a number.')
        return False
    try:
        int(quantity)
```

```

        if int(quantity) < 0:
            messagebox.showerror('Error', 'Quantity must be a positive
integer.')
```

 return False

```

    except ValueError:
        messagebox.showerror('Error', 'Quantity must be an integer.')
```

 return False

```

    if status == 'Select':
        messagebox.showerror('Error', 'Please select a Status.')
```

 return False

```

    return True
```

def validate_input(category, supplier, name, price, quantity, status):

 """Validates the input fields."""

```

    if category == 'Select':
        messagebox.showerror('Error', 'Please select a Category.')
```

 return False

```

    if supplier == 'Select':
        messagebox.showerror('Error', 'Please select a Supplier.')
```

 return False

```

    if not name:
        messagebox.showerror('Error', 'Please enter a Product Name.')
```

 return False

```

    try:
        float(price)
```

except ValueError:

```

    messagebox.showerror('Error', 'Price must be a number.')
```

 return False

```

    try:
        int(quantity)
```

 if int(quantity) < 0:

```

            messagebox.showerror('Error', 'Quantity must be a positive
integer.')
```

 return False

except ValueError:

```

    messagebox.showerror('Error', 'Quantity must be an integer.')
```

 return False

```

    if status == 'Select':
        messagebox.showerror('Error', 'Please select a Status.')
```

 return False

```

    return True
```

Add the following functions outside the `product\_form` function:

```

def product_form(window):
    global back_image
    product_frame=Frame(window,width=1070,height=567,bg='white')
    product_frame.place(x=200,y=100)
```

```

back_image = PhotoImage(file='arrow.png')
back_button = Button(product_frame, image=back_image, bd=0,
cursor='hand2', bg='white', command=lambda: product_frame.place_forget())
back_button.place(x=10, y=10)

left_frame=Frame(product_frame,bg='white',bd=2,relief=RIDGE)
left_frame.place(x=20,y=40)

heading_Label = Label(left_frame, text='Manage Products Details',
font=('times new roman', 16, 'bold'), bg='#0f4d7d', fg='white')
heading_Label.grid(row=0,columnspan=2,sticky="we")

category_label = Label(left_frame, text='Category:', font=('times new
roman', 14, 'bold'), bg='white')
category_label.grid(row=1, column=0, padx=20, sticky="w")
category_combobox=ttk.Combobox(left_frame,font=('times new roman',
14),width=20,state='readonly')
category_combobox.grid(row=1,column=1,pady=30)
category_combobox.set('Select')

supplier_label = Label(left_frame, text='Supplier:', font=('times new
roman', 14, 'bold'), bg='white')
supplier_label.grid(row=2, column=0, padx=20, sticky="w")
supplier_combobox=ttk.Combobox(left_frame,font=('times new roman',
14),width=20,state='readonly')
supplier_combobox.grid(row=2,column=1)
supplier_combobox.set('Select')

name_label = Label(left_frame, text='Product Name:', font=('times new
roman', 14, 'bold'), bg='white')
name_label.grid(row=3, column=0, padx=20, sticky="w")
name_entry = Entry(left_frame, font=('times new roman', 14, 'bold'),
bg='white')
name_entry.grid(row=3, column=1,pady=30)

price_label = Label(left_frame, text='Price:', font=('times new roman',
14, 'bold'), bg='white')
price_label.grid(row=4, column=0, padx=20, sticky="w")
price_entry = Entry(left_frame, font=('times new roman', 14, 'bold'),
bg='white')
price_entry.grid(row=4, column=1)

quantity_label = Label(left_frame, text='Quantity:', font=('times new
roman', 14, 'bold'), bg='white')
quantity_label.grid(row=5, column=0, padx=20, sticky="w")
quantity_entry = Entry(left_frame, font=('times new roman', 14, 'bold'),
bg='white')
quantity_entry.grid(row=5, column=1,pady=30)

```

```

        status_label = Label(left_frame, text='Status:', font=('times new roman',
14, 'bold'), bg='white')
        status_label.grid(row=6, column=0, padx=20, sticky="w")
        status_combobox=ttk.Combobox(left_frame,values=('Active','Inactive'),font=
('times new roman', 14),width=20,state='readonly')
        status_combobox.grid(row=6,column=1)
        status_combobox.set('Select')

        button_frame=Frame(left_frame,bg="white")
        button_frame.grid(row=7,columnspan=2,padx=20)

        add_button = Button(button_frame,text='Add',font=('times new roman',
14),width=8,cursor='hand2',fg='white',bg='#0f4d7d',command=lambda:
add_product(category_combobox.get(),supplier_combobox.get(),name_entry.get(),p
rice_entry.get(),quantity_entry.get(),status_combobox.get(),treeview))
        add_button.grid(row=0, column=0, padx=5)

        update_button = Button(button_frame, text='Update', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',command=lambda:
update_product(category_combobox.get(), supplier_combobox.get(),
name_entry.get(),
                                                    price_entry.get(),
quantity_entry.get(), status_combobox.get(), treeview))
        update_button.grid(row=0, column=1, padx=5)

        delete_button = Button(button_frame, text='Delete', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                                command=lambda: delete_product(treeview))
        delete_button.grid(row=0, column=2, padx=5, pady=20)

        clear_button = Button(button_frame, text='Clear', font=('times new roman',
14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                                command=lambda: clear_form(category_combobox,
supplier_combobox, name_entry, price_entry, quantity_entry, status_combobox))
        clear_button.grid(row=0, column=3, padx=5, pady=20)

        search_frame=LabelFrame(product_frame,text='Search Product',font=('times
new roman',14,'bold'), bg='white')
        search_frame.place(x=480,y=30)

        search_combobox=ttk.Combobox(search_frame,values=('Category','Supplier','N
ame','Status'),state='readonly',width=16,font=('times new roman',14,'bold'))
        search_combobox.grid(row=0,column=0,padx=10)
        search_combobox.set('Search By')

        search_entry = Entry(search_frame, font=('times new roman', 14, 'bold'),
bg='white',width=16)

```

```

search_entry.grid(row=0, column=1)

search_button = Button(search_frame, text='Search', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                      command=lambda: search_product(treeview,
search_combobox.get(), search_entry.get()))
search_button.grid(row=0, column=2, padx=(10, 0), pady=10)

show_button = Button(search_frame, text='Show all', font=('times new
roman', 14), width=8, cursor='hand2', fg='white', bg='#0f4d7d',
                    command=lambda: show_all_products(treeview))
show_button.grid(row=0, column=3, padx=10)

treeview_frame=Frame(product_frame)
treeview_frame.place(x=480,y=125,width=570,height=410)

scrolly = Scrollbar(treeview_frame, orient=VERTICAL)
scrollx = Scrollbar(treeview_frame, orient=HORIZONTAL)
treeview = ttk.Treeview(treeview_frame, column=('id','category',
'supplier', 'name','price','quantity','status'), show='headings',
                      yscrollcommand=scrolly.set,
xscrollcommand=scrollx.set)
scrolly.pack(side=RIGHT, fill=Y)
scrollx.pack(side=BOTTOM, fill=X)
scrollx.config(command=treeview.xview)
scrolly.config(command=treeview.yview)
treeview.pack(fill=BOTH, expand=1)

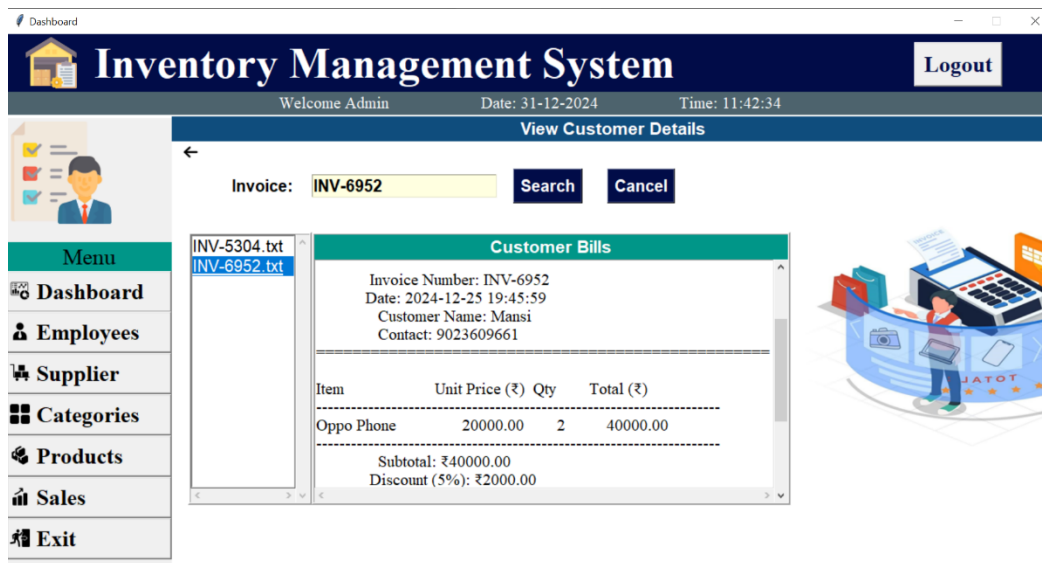
treeview.heading('id', text='Id')
treeview.heading('category', text='Category')
treeview.heading('supplier', text='Supplier')
treeview.heading('name', text='Name')
treeview.heading('price', text='Price')
treeview.heading('quantity', text='Quantity')
treeview.heading('status', text='Status')

treeview.column('id', width=80)
treeview.column('category', width=160)
treeview.column('supplier', width=160)
treeview.column('name', width=160)
treeview.column('price', width=160)
treeview.column('quantity', width=160)
treeview.column('status', width=160)
treeview.bind(
    "<ButtonRelease-1>",
    lambda event: select_item(
        treeview, category_combobox, supplier_combobox,
        name_entry, price_entry, quantity_entry, status_combobox

```

```
    )  
)  
  
# Fetch initial data for the Treeview  
fetch_supplier_category(category_combobox, supplier_combobox)  
treeview_data(treeview)
```

6.1.7 Manage Sales



Code:

```
from tkinter import *
from PIL import Image, ImageTk
from tkinter import messagebox
import os

def show(window, sales_list):
    sales_list.delete(0, END)
    # List files in 'bills' directory and filter for '.txt' files
    for i in os.listdir('bills'):
        if i.split('.')[-1] == 'txt': # Only show .txt files
            sales_list.insert(END, i)

def get_data(event, bill_area):
    # Get the widget from the event
    sales_list = event.widget
    # Get the selected index
    index_ = sales_list.curselection() # Check if a selection is made
    filename = sales_list.get(index_[0])
    print("Selected file:", filename)
    bill_area.delete('1.0', END)
    filepath = f'bills/{filename}' # Update to correct path
    try:
        with open(filepath, 'r', encoding='utf-8') as fp:
            bill_area.insert(END, fp.read())
    except Exception as e:
        messagebox.showerror("Error", f"Could not open the bill: {e}")
```



```

def sales_form(window):
    global back_image, bill_photo

    sales_frame = Frame(window, width=1070, height=567, bg='white')
    sales_frame.place(x=200, y=100)

    heading_Label = Label(sales_frame, text='View Customer Details',
font=('Arial Unicode MS', 16, 'bold'), bg='#0f4d7d', fg='white')
    heading_Label.place(x=0, y=0, relwidth=1)

    back_image = PhotoImage(file='arrow.png')
    back_button = Button(sales_frame, image=back_image, bd=0, cursor='hand2',
bg='white', command=lambda: sales_frame.place_forget())
    back_button.place(x=10, y=30)

    detail_frame = Frame(sales_frame, bg='white')
    detail_frame.place(x=50, y=50)

    id_label = Label(detail_frame, text='Invoice:', font=('Arial Unicode MS',
15, 'bold'), bg='white')
    id_label.grid(row=0, column=0, padx=20, sticky="w")
    id_entry = Entry(detail_frame, font=('Arial Unicode MS', 15, 'bold'),
bg='lightyellow')
    id_entry.grid(row=0, column=1, pady=20)

    # Search Button
    def search_invoice():
        invoice_id = id_entry.get().strip()
        if not invoice_id:
            messagebox.showerror("Error", "Please enter an invoice ID!")
            return

        # Check if the invoice file exists in the 'bills' directory
        filepath = f'bills/{invoice_id}.txt'
        if os.path.exists(filepath):
            with open(filepath, 'r') as fp:
                bill_area.delete('1.0', END)
                bill_area.insert(END, fp.read())
        else:
            messagebox.showinfo("Not Found", f"Invoice '{invoice_id}' not
found!")

    btn_search = Button(
        detail_frame,
        text="Search",
        font=('Arial Unicode MS', 15, 'bold'),
        bg="#010c48",
        fg="white",

```

```

        cursor="hand2",
        command=search_invoice
    )
    btn_search.grid(row=0, column=2, padx=20)

    # Cancel Button
    def cancel_search():
        id_entry.delete(0, END) # Clear the invoice entry field
        bill_area.delete('1.0', END) # Clear the bill display area
        show(window, sales_list) # Reload the sales list

    btn_cancel = Button(
        detail_frame,
        text="Cancel",
        font=('Arial Unicode MS', 15, 'bold'),
        bg="#010c48",
        fg="white",
        cursor="hand2",
        command=cancel_search
    )
    btn_cancel.grid(row=0, column=3, padx=10)

    salesd_frame = Frame(sales_frame, bd=3, relief=RIDGE)
    salesd_frame.place(x=20, y=140, width=150, height=330)

    # Adding both vertical and horizontal scrollbars for sales_list
    scrolly = Scrollbar(salesd_frame, orient=VERTICAL)
    scrollx = Scrollbar(salesd_frame, orient=HORIZONTAL)
    sales_list = Listbox(salesd_frame, font=('Arial Unicode MS', 15),
bg="white", yscrollcommand=scrolly.set, xscrollcommand=scrollx.set)
    scrolly.pack(side=RIGHT, fill=Y)
    scrollx.pack(side=BOTTOM, fill=X) # Position the horizontal scrollbar at
the bottom
    scrolly.config(command=sales_list.yview)
    scrollx.config(command=sales_list.xview)
    sales_list.pack(fill=BOTH, expand=1)

    # Bind the listbox selection to the get_data function
    sales_list.bind("<ButtonRelease-1>", lambda event: get_data(event,
bill_area))

    bill_frame = Frame(sales_frame, bd=3, relief=RIDGE)
    bill_frame.place(x=170, y=140, width=580, height=330)

    heading_Label = Label(bill_frame, text='Customer Bills', font=('Arial
Unicode MS', 16, 'bold'), bg='#009688', fg='white')
    heading_Label.pack(side=TOP, fill=X)

```

```

# Adding both vertical and horizontal scrollbars for bill_area
scrolly2 = Scrollbar(bill_frame, orient=VERTICAL)
scrollx2 = Scrollbar(bill_frame, orient=HORIZONTAL)
bill_area = Text(bill_frame, font=('times new roman', 15), bg="white",
yscrollcommand=scrolly2.set, xscrollcommand=scrollx2.set, wrap="word")
scrolly2.pack(side=RIGHT, fill=Y)
scrollx2.pack(side=BOTTOM, fill=X) # Position the horizontal scrollbar at
the bottom
scrolly2.config(command=bill_area.yview)
scrollx2.config(command=bill_area.xview)
bill_area.pack(fill=BOTH, expand=1)

bill_photo = Image.open('logobill.jpg') # Open the image
bill_photo = bill_photo.resize((450, 300)) # Resize the image
bill_photo = ImageTk.PhotoImage(bill_photo)

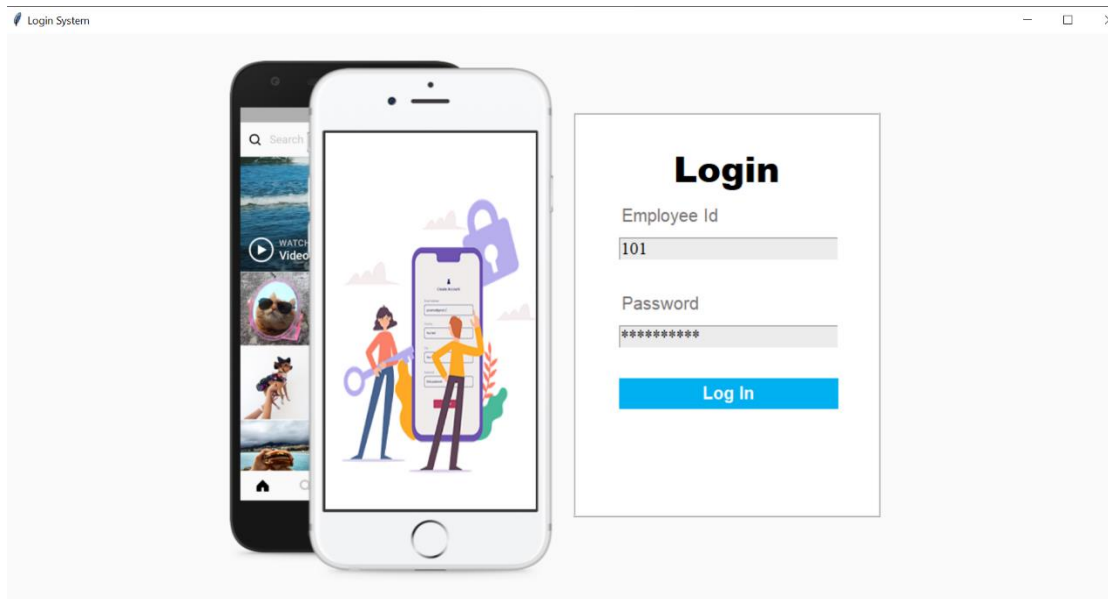
lbl_image = Label(sales_frame, image=bill_photo, bd=0)
lbl_image.image = bill_photo # Persist the image reference
lbl_image.place(x=755, y=110)

show(window, sales_list) # Pass `sales_list` to `show` to load the bills

```

6.2 Employee Side

6.2.1 Login Page



Code:

```
from tkinter import *
from PIL import ImageTk
from tkinter import messagebox
import mysql.connector
import os

class Login_System:
    def __init__(self,root):
        self.root=root
        self.root.title("Login System")
        self.root.geometry("1350x700+0+0")
        self.root.config(bg="#fafafa")

        self.phone_image=ImageTk.PhotoImage(file="phone.png")
        self.lbl_Phone_image=Label(self.root,image=self.phone_image,bd=0).place(x=200,y=10)

        self.employee_id=StringVar()
        self.password=StringVar()

        login_frame=Frame(self.root,bd=2,relief=RIDGE,bg="white")
        login_frame.place(x=650,y=90,width=350,height=460)

        title=Label(login_frame,text="Login", font=("Arial
Black",30,"bold"),bg="white").place(x=0,y=30,relwidth=1)
```

```

        lbl_user=Label(login_frame,text="Employee
Id",font=("Andalus",15),bg="white",fg="#767171").place(x=50,y=100)
        txt_employee_id=Entry(login_frame,textvariable=self.employee_id,font=(
'times new roman',15),bg="#ecec").place(x=50,y=140,width=250)

        lbl_pass=Label(login_frame,text="Password",font=("Andalus",15),bg="whi
te",fg="#767171").place(x=50,y=200)
        txt_pass=Entry(login_frame,textvariable=self.password,show="*",font=(
'times new roman',15),bg="#ecec").place(x=50,y=240,width=250)

        btn_login=Button(login_frame,text="Log
In",command=self.login,font=("Arial Rounded MT
Bold",15,"bold"),bg="#00b0f0",fg="white",activebackground="#00b0f0",activefore
ground="white",bd=0,cursor="hand2").place(x=50,y=300,width=250,height=35)

        """"hr=Label(login_frame,bg="lightgray").place(x=50,y=370,width=250,hei
ght=2)
        or_=Label(login_frame,text="OR",bg="white",fg="lightgray",font=("times
new roman",15,"bold")).place(x=160,y=355)""""

        #btn_forget=Button(login_frame,text="Forget
Password?",command=self.forget_window,font=("times new
roman",13),bg="white",fg="#00759e",activebackground="white",activeforeground="
#00759e",bd=0,cursor="hand2").place(x=110,y=390)

        self.im1=ImageTk.PhotoImage(file="im1.png")
        self.im2=ImageTk.PhotoImage(file="im2.png")
        self.im3=ImageTk.PhotoImage(file="im3.png")

        self.lbl_change_image=Label(self.root,bg="gray")
        self.lbl_change_image.place(x=367,y=113,width=240,height=428)

        self.animate()

    def animate(self):
        self.im=self.im1
        self.im1=self.im2
        self.im2=self.im3
        self.im3=self.im
        self.lbl_change_image.config(image=self.im)
        self.lbl_change_image.after(1500,self.animate)

root=Tk()
obj=Login_System(root)
root.mainloop()

```

6.2.2 Employee Dashboard

Inventory Management System

Welcome Admin Date: 15-11-2024 Time: 9:00:00pm

All Products

Search Product | By Name Show all

Product Name: LG Search

PID	Name	Price	Quantity
2	LG Fridge	20000.00	21
3	LG TV	20000.00	23

Note: Enter 0 Quantity to remove product from the cart

Customer Details

Name: Mansi Contact: 9023609661

Cart Total Products: [1]

PID	Name	Pri
3	LG TV	20000.00

Product Name: LG TV Price Per Qty: 20000.00 Quantity: 2

In Stock: [23] Clear Add

Customer Bill Area

ELECTRONICS SHOP BILLING SYSTEM
*** OFFICIAL BILL ***

Invoice Number: INV-5013
Date: 2024-12-31 11:44:08
Customer Name: Mansi
Contact: 9023609661

Item	Unit Price (₹)	Qty	Total (₹)
LG TV	20000.00	2	40000.00

Subtotal: ₹40000.00
Discount (5%): ₹2000.00
Net Payable: ₹38000.00

Thank you for shopping with us!
We look forward to serving you again.
For inquiries, contact us at support@shop.com

Bill Amount ₹40000.00	Discount ₹2000.00	Net Pay ₹38000.00
--------------------------	----------------------	----------------------

Print Clear All Generate/Save Bill

Code:

```
import tkinter as tk
from tkinter import *
from PIL import ImageTk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry
import mysql.connector
from datetime import date
import datetime
from datetime import datetime
import os
from tkinter import filedialog

import os
#GUI Part
# Create a Tkinter window
window = Tk()
window.title("Dashboard")
window.geometry("1270x668+0+0")
window.resizable(0, 0)
window.config(bg='white')
def redirect_to_login():
    window.destroy() # Close the current window
    os.system('python logine.py')
# Load background image
bg_image = PhotoImage(file='inventory.png')
```

```

# Title label with image and text
titleLabel = Label(window, image=bg_image, compound=LEFT, text="Inventory
Management System",
                    font=("times new roman", 40, "bold"), bg="#010c48",
                    fg="white", anchor="w", padx=20)
titleLabel.place(x=0, y=0, relwidth=1)
cart_list=[]

logoutButton = Button(window, text="Logout", font=('times new roman', 20,
'bold'), fg='#010c48', command=redirect_to_login, cursor="hand2")
logoutButton.place(x=1100, y=10)

subtitleLabel=Label(window, text="Welcome Admin\t\t Date:15-11-2024\t\t
Time:9:00:00pm", font=('times new roman', 15), bg='#4d636d', fg='white')
subtitleLabel.place(x=0, y=70, relwidth=1)

varsearch=StringVar()
product_frame=Frame(window, bd=4, relief=RIDGE, bg="white")
product_frame.place(x=5, y=110, width=390, height=540)
pTitle=Label(product_frame, text="All Products", font=('times new
roman', 20, 'bold'), bg="#262626", fg="white").pack(side=TOP, fill=X)

product_frame2=Frame(product_frame, relief=RIDGE, bg="white")
product_frame2.place(x=1, y=38, width=380, height=90)

lblsearch=Label(product_frame2, text="Search Product | By Name ", font=('times
new roman', 15, 'bold'), bg="white", fg="green").place(x=2, y=5)

lblname=Label(product_frame2, text="Product Name ", font=('times new
roman', 15, 'bold'), bg="white").place(x=5, y=45)
txtsearch=Entry(product_frame2, textvariable=varsearch, font=('times new
roman', 15), bg="white").place(x=130, y=47, width=140, height=22)
# ... other code

# Create the button
btnsearch = Button(product_frame2, text="Search", font=('times new roman',
15), bg="#219653", fg="white", cursor="hand2")

# Place the button
btnsearch.place(x=274, y=45, width=100, height=25)

# Configure the command (this is the crucial change)
btnsearch.config(command=lambda:search_products)

# ... rest of your code
# ... other code

```

```

# Create the button
btnshowall = Button(product_frame2, text="Show all", font=('times new roman',
15), bg="#219653", fg="white", cursor="hand2")

# Place the button
btnshowall.place(x=274, y=10, width=100, height=25)

# **Directly assign function**
btnshowall.config(command=lambda: show_all_products())
# ... rest of your code
product_frame3=Frame(product_frame,relief=RIDGE)
product_frame3.place(x=2,y=140,width=380,height=390)

scrolly=Scrollbar(product_frame3,orient=VERTICAL)
scrollx=Scrollbar(product_frame3,orient=HORIZONTAL)

product_table=ttk.Treeview(product_frame3,columns=("pid","name","price","qty",
"status"),yscrollcommand=scrolly.set,xscrollcommand=scrollx.set)
scrollx.pack(side=BOTTOM,fill=X)
scrolly.pack(side=RIGHT,fill=Y)
scrollx.config(command=product_table.xview)
scrolly.config(command=product_table.yview)
def get_data(ev):
    selected_row = product_table.focus()
    if selected_row: # Check if a row is actually selected
        content = product_table.item(selected_row)
        row = content['values']
        var_pid.set(row[0])
        var_pname.set(row[1])
        var_price.set(row[2])
        # Check if quantity information exists (at least 4 elements)
        if len(row) >= 4:
            lbl_inStock.config(text=f"In Stock [{row[3]}]")
        else:
            lbl_inStock.config(text="In Stock [Data Missing]") # Handle
missing data
    else:
        # Handle the case where no row is selected
        var_pid.set("")
        var_pname.set("")
        var_price.set("")
        lbl_inStock.config(text="In Stock [-]") # Clear display or show
default message

product_table.heading("pid",text="PID")
product_table.heading("name",text="Name")
product_table.heading("price",text="Price")
product_table.heading("qty",text="Quantity")

```



```

product_table.heading("status",text="Status")
product_table["show"]="headings"
product_table.column("pid",anchor=tk.CENTER,width=90)
product_table.column("name",anchor=tk.CENTER,width=100)
product_table.column("price",anchor=tk.CENTER,width=100)
product_table.column("qty",anchor=tk.CENTER,width=100)
product_table.column("status",anchor=tk.CENTER,width=100)
product_table.pack(fill=BOTH,expand=1)
product_table.bind("<ButtonRelease-1>", get_data)

lbl_note=Label(product_frame3,text="Note:Enter 0 Quantity to remove product
from the cart",font=('times new
roman',12),bg="white",fg="red").pack(side=BOTTOM,fill=X)

customer_frame=Frame(window,bd=4,relief=RIDGE,bg="white")
customer_frame.place(x=395,y=110,width=490,height=70)

pTitle=Label(customer_frame,text="Customer Details",font=('times new
roman',15,'bold'),bg="lightgray").pack(side=TOP,fill=X)

var_cname=StringVar()
var_contact=StringVar()
lblname=Label(customer_frame,text="Name ",font=('times new
roman',15),bg="white").place(x=5,y=35)
txtsearch=Entry(customer_frame,textvariable=var_cname,font=('times new
roman',13),bg="white").place(x=60,y=35,width=180,height=22)

lblcontact=Label(customer_frame,text="Contact",font=('times new
roman',15),bg="white").place(x=230,y=35)
txtcontact=Entry(customer_frame,textvariable=var_contact,font=('times new
roman',13),bg="white").place(x=300,y=35,width=180,height=22)

cal_cart_Frame=Frame(window,bd=4,relief=RIDGE,bg="white")
cal_cart_Frame.place(x=395,y=180,width=490,height=340)

var_cal_input=StringVar()
cal_Frame=Frame(cal_cart_Frame,relief=RIDGE,bg="white")
cal_Frame.place(x=5,y=5,width=268,height=320)

txt_cal_input=Entry(cal_Frame,textvariable=var_cal_input,font=('arial',15,'bol
d'),width=23,bd=8,relief=GROOVE,state='readonly',justify=RIGHT)
txt_cal_input.grid(row=0,columnspan=4)

btn_7=Button(cal_Frame,text='7',font=('arial',15,'bold'),command=lambda:get_in
put(7),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=1,column=0)
btn_8=Button(cal_Frame,text='8',font=('arial',15,'bold'),command=lambda:get_in
put(8),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=1,column=1)

```

```

btn_9=Button(cal_Frame,text='9',font=('arial',15,'bold'),command=lambda:get_in
put(9),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=1,column=2)
btn_sum=Button(cal_Frame,text='+',font=('arial',15,'bold'),command=lambda:get_
input('+'),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=1,column=3)

btn_4=Button(cal_Frame,text='4',font=('arial',15,'bold'),command=lambda:get_in
put(4),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=2,column=0)
btn_5=Button(cal_Frame,text='5',font=('arial',15,'bold'),command=lambda:get_in
put(5),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=2,column=1)
btn_6=Button(cal_Frame,text='6',font=('arial',15,'bold'),command=lambda:get_in
put(6),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=2,column=2)
btn_sub=Button(cal_Frame,text='-
',font=('arial',15,'bold'),command=lambda:get_input('-
'),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=2,column=3)

btn_1=Button(cal_Frame,text='1',font=('arial',15,'bold'),command=lambda:get_in
put(1),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=3,column=0)
btn_2=Button(cal_Frame,text='2',font=('arial',15,'bold'),command=lambda:get_in
put(2),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=3,column=1)
btn_3=Button(cal_Frame,text='3',font=('arial',15,'bold'),command=lambda:get_in
put(3),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=3,column=2)
btn_mul=Button(cal_Frame,text='*',font=('arial',15,'bold'),command=lambda:get_
input('*'),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=3,column=3)

btn_0=Button(cal_Frame,text='0',font=('arial',15,'bold'),command=lambda:get_in
put(0),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=4,column=0)
btn_c=Button(cal_Frame,text='C',font=('arial',15,'bold'),command=lambda:clear_
cal(),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=4,column=1)
btn_eq=Button(cal_Frame,text='=',font=('arial',15,'bold'),command=lambda:perfo
rm_cal(),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=4,column=2)
btn_div=Button(cal_Frame,text='/',font=('arial',15,'bold'),command=lambda:get_
input('/'),bd=6,width=4,height=1,pady=10,cursor="hand2").grid(row=4,column=3)

cart_frame=Frame(cal_cart_Frame,relief=RIDGE,bd=0)
cart_frame.place(x=275,y=5,width=205,height=320)
cartTitle = Label(cart_frame, text="Cart \t Total Products[0]", font=('times
new roman', 13), bg="lightgray")
cartTitle.pack(side=TOP, fill=X)

scrolly=Scrollbar(cart_frame,orient=VERTICAL)
scrollx=Scrollbar(cart_frame,orient=HORIZONTAL)

cart_table=ttk.Treeview(cart_frame,columns=("pid","name","price","qty","status
"),yscrollcommand=scrolly.set,xscrollcommand=scrollx.set)
scrollx.pack(side=BOTTOM,fill=X)
scrolly.pack(side=RIGHT,fill=Y)
scrollx.config(command=cart_table.xview)
scrolly.config(command=cart_table.yview)

```

```

cart_table.heading("pid",text="PID")
cart_table.heading("name",text="Name")
cart_table.heading("price",text="Price")
cart_table.heading("qty",text="Quantity")
cart_table.heading("status",text="Status")
cart_table["show"]="headings"
cart_table.column("pid",width=40)
cart_table.column("name",width=100)
cart_table.column("price",width=90)
cart_table.column("qty",width=40)
cart_table.column("status",width=90)
cart_table.pack(fill=BOTH,expand=1)

var_pid=StringVar()
var_pname=StringVar()
var_price=StringVar()
var_qty=StringVar()
var_stock=StringVar()

Add_CartWidgetsFrame=Frame(window,bd=2,relief=RIDGE,bg="white")
Add_CartWidgetsFrame.place(x=395,y=510,width=490,height=120)

lbl_p_name=Label(Add_CartWidgetsFrame,text="Product Name",font=("times new
roman",15),bg="white").place(x=5,y=5)
txt_p_name=Entry(Add_CartWidgetsFrame,textvariable=var_pname,font=("times new
roman",15),bg="lightyellow",state='readonly').place(x=5,y=35,width=130,height=
22)

lbl_p_price=Label(Add_CartWidgetsFrame,text="Price Per Qty",font=("times new
roman",15),bg="white").place(x=160,y=5)
txt_p_price=Entry(Add_CartWidgetsFrame,textvariable=var_price,font=("times new
roman",15),bg="lightyellow",state='readonly').place(x=160,y=35,width=130,height=
22)

lbl_p_qty=Label(Add_CartWidgetsFrame,text="Quantity",font=("times new
roman",15),bg="white").place(x=315,y=5)
txt_p_qty=Entry(Add_CartWidgetsFrame,textvariable=var_qty,font=("times new
roman",15),bg="lightyellow").place(x=315,y=35,width=130,height=22)

lbl_inStock = Label(Add_CartWidgetsFrame, text="In Stock [9999]", font=("times
new roman", 15), bg="white")
lbl_inStock.place(x=5, y=65)
btn_clear_cart=Button(Add_CartWidgetsFrame,text="Clear",font=("times new
roman",15,"bold"),bg="lightgray",cursor="hand2", command=lambda:
clear_cart_widgets()).place(x=160,y=65,width=130,height=30)
btn_add_cart = Button(Add_CartWidgetsFrame,
text="Add",

```

```

        command=lambda: add_update_cart(),
        font=("times new roman", 15, "bold"),
        bg="orange",
        cursor="hand2")
btn_add_cart.place(x=315, y=65, width=130, height=30)
billFrame=Frame(window,bd=2,relief=RIDGE,bg='white')
billFrame.place(x=885,y=110,width=380,height=410)

BTitle=Label(billFrame,text="Customer Bill Area",font=('times new
roman',15,'bold'),bg="#262626",fg="white").pack(side=TOP,fill=X)
scrolly=Scrollbar(billFrame,orient=VERTICAL)
scrolly.pack(side=RIGHT,fill=Y)
txt_bill_area=Text(billFrame,yscrollcommand=scrolly.set,font=('times new
roman',10))
txt_bill_area.pack(fill=BOTH,expand=1)
scrolly.config(command=txt_bill_area)

billMenuFrame=Frame(window,bd=2,relief=RIDGE,bg='white')
billMenuFrame.place(x=885,y=520,width=380,height=140)

lbl_amnt=Label(billMenuFrame,text='Bill Amount\n[0]',font=('times new
roman',12,'bold'),bg="#3f51b5",fg="white")
lbl_amnt.place(x=2,y=3,width=120,height=65)

lbl_discount=Label(billMenuFrame,text='Discount\n[5%]',font=('times new
roman',12,'bold'),bg="#8bc34a",fg="white")
lbl_discount.place(x=124,y=3,width=120,height=65)

lbl_net_pay=Label(billMenuFrame,text='Net Pay\n[0]',font=('times new
roman',12,'bold'),bg="#607d8b",fg="white")
lbl_net_pay.place(x=246,y=3,width=140,height=65)

btn_print=Button(billMenuFrame,text='Print',cursor='hand2',font=('times new
roman',12,'bold'),bg="red",fg="white")
btn_print.place(x=2,y=70,width=120,height=50)

btn_clear_all=Button(billMenuFrame,text='Clear
All',cursor='hand2',font=('times new
roman',12,'bold'),bg="gray",fg="white",command=lambda:clear_allb)
btn_clear_all.place(x=124,y=70,width=120,height=50)

btn_generate = Button(billMenuFrame,
        text='Generate/Save Bill',
        cursor='hand2',
        font=('times new roman', 12, 'bold'),
        bg="#009688",
        fg="white",
        command=lambda:generate_bill) # Assign the function here

```

```

btn_generate.place(x=246, y=70, width=140, height=50)

def get_input(num):
    xnum=var_cal_input.get()+str(num)
    var_cal_input.set(xnum)
def clear_cal():
    var_cal_input.set('')

def perform_cal():
    result=var_cal_input.get()
    var_cal_input.set(eval(result))

def generate_bill():
    """Generates a professionally formatted, center-aligned bill, decreases
product quantities,
stores it in the database, and saves it as a .txt file."""
    from datetime import datetime
    import mysql.connector
    from tkinter import messagebox
    import tkinter as tk
    import random
    import os

    try:
        # Database connection
        db = mysql.connector.connect(
            host="localhost",
            user="root",
            password="",
            database="inventory_system"
        )
        cursor = db.cursor()

        # Current date and time
        current_date = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

        # Generate a random invoice number
        invoice_number = f"INV-{random.randint(1000, 9999)}"

        # Retrieve customer details
        customer_name = var_cname.get().strip()
        customer_contact = var_contact.get().strip()

        # Validate customer name and contact
        if not customer_name:
            messagebox.showerror("Error", "Customer name is required",
parent=window)
            return

```

```

        if not customer_contact:
            messagebox.showerror("Error", "Contact number is required",
parent=window)
            return
        if not customer_contact.isdigit() or len(customer_contact) != 10:
            messagebox.showerror("Error", "Enter a valid 10-digit contact
number", parent=window)
            return

        # Check if cart is empty
        cart_items = cart_table.get_children()
        if not cart_items:
            messagebox.showerror("Error", "Cart is empty! Please add items
before generating a bill.", parent=window)
            return

        # Calculate total amount and prepare item data
        total_amount = 0.0
        item_data = []
        for item in cart_items:
            item_values = cart_table.item(item)['values']
            item_id = item_values[0] # Assuming the first value is the
product ID
            item_name = item_values[1]
            item_price = float(item_values[2])
            item_qty = int(item_values[3])
            item_total = item_price * item_qty
            total_amount += item_total
            item_data.append((item_id, item_name, item_price, item_qty,
item_total))

        # Apply discount
        discount_rate = 0.05 # 5% discount
        discount_amount = total_amount * discount_rate
        net_payable = total_amount - discount_amount

        # Insert bill data into the `bills` table
        cursor.execute(
            "INSERT INTO bills (invoice_number, customer_name,
customer_contact, bill_date, subtotal, discount_amount, net_payable) "
            "VALUES (%s, %s, %s, %s, %s, %s, %s)",
            (invoice_number, customer_name, customer_contact, current_date,
total_amount, discount_amount, net_payable)
        )
        bill_id = cursor.lastrowid

        # Insert items into the `bill_items` table and update product
quantities

```

```

    for item in item_data:
        product_id = item[0]
        item_name = item[1]
        item_price = item[2]
        item_qty = item[3]
        item_total = item[4]

        # Insert into `bill_items`
        cursor.execute(
            "INSERT INTO bill_items (bill_id, item_name, item_price,
item_qty, item_total) VALUES (%s, %s, %s, %s, %s)",
            (bill_id, item_name, item_price, item_qty, item_total)
        )

        # Update product quantity
        cursor.execute(
            "UPDATE product_data SET quantity = quantity - %s WHERE id =
%s AND quantity >= %s",
            (item_qty, product_id, item_qty)
        )
        if cursor.rowcount == 0:
            db.rollback()
            messagebox.showerror("Error", f"Insufficient stock for product
ID: {product_id}", parent=window)
            return

    db.commit()

    # Generate the bill text
    bill_width = 50
    center = lambda text: text.center(bill_width)

    bill_text = "\n"
    bill_text += "=" * bill_width + "\n"
    bill_text += center("ELECTRONICS SHOP BILLING SYSTEM") + "\n"
    bill_text += center("*** OFFICIAL BILL ***") + "\n"
    bill_text += "=" * bill_width + "\n\n"
    bill_text += f"{center(f'Invoice Number: {invoice_number}')} \n"
    bill_text += f"{center(f'Date: {current_date}')} \n"
    bill_text += f"{center(f'Customer Name: {customer_name}')} \n"
    bill_text += f"{center(f'Contact: {customer_contact}')} \n"
    bill_text += "=" * bill_width + "\n\n"

    # Table header
    bill_text += center("{:<25} {:<15} {:<10} {:<15}".format("Item", "Unit
Price (₹)", "Qty", "Total (₹)")) + "\n"
    bill_text += center("-" * 70) + "\n"

```

```

# Table content
for item in item_data:
    item_line = "{:<25} {:<15.2f} {:<10}
{:<15.2f}".format(item[1][:25], item[2], item[3], item[4])
    bill_text += center(item_line) + "\n"

bill_text += center("-" * 70) + "\n"

# Summary
bill_text += f"{center(f'Subtotal: ₹{total_amount:.2f}')} \n"
bill_text += f"{center(f'Discount (5%): ₹{discount_amount:.2f}')} \n"
bill_text += f"{center(f'Net Payable: ₹{net_payable:.2f}')} \n"
bill_text += "=" * bill_width + "\n\n"

# Footer
bill_text += center("Thank you for shopping with us!") + "\n"
bill_text += center("We look forward to serving you again.") + "\n"
bill_text += center("For inquiries, contact us at support@shop.com") +
"\n"

bill_text += "=" * bill_width + "\n"

# Display the bill
txt_bill_area.delete("1.0", tk.END)
txt_bill_area.insert("1.0", bill_text)

# Save the bill as a .txt file
save_path = os.path.join(os.getcwd(), "bills") # Save in a "bills"
directory
os.makedirs(save_path, exist_ok=True)
file_name = f"{invoice_number}.txt"
file_path = os.path.join(save_path, file_name)
with open(file_path, "w", encoding="utf-8") as file:
    file.write(bill_text)

messagebox.showinfo("Saved", f"Bill saved successfully as
{file_name}", parent=window)

# Update summary labels
lbl_amnt.config(text=f"Bill Amount\n₹{total_amount:.2f}")
lbl_discount.config(text=f"Discount\n₹{discount_amount:.2f}")
lbl_net_pay.config(text=f"Net Pay\n₹{net_payable:.2f}")

except mysql.connector.Error as db_error:
    messagebox.showerror("Database Error", f"Error interacting with the
database: {str(db_error)}", parent=window)
except Exception as e:
    messagebox.showerror("Error", f"An error occurred: {str(e)}",
parent=window)

```



```

finally:
    if 'db' in locals() and db.is_connected():
        cursor.close()
        db.close()

def clear_allb():
    """Clears all bill-related information."""

    # Clear bill area text
    txt_bill_area.delete("1.0", tk.END)

    # Clear cart
    for item in cart_table.get_children():
        cart_table.delete(item)
    cartTitle.config(text="Cart \t Total Products[0]")

    # Clear customer details
    var_cname.set("")
    var_contact.set("")

    # Clear bill amount labels
    lbl_amnt.config(text='Bill Amount\n[0]')
    lbl_discount.config(text='Discount\n[5%]')
    lbl_net_pay.config(text='Net Pay\n[0]')

# Assuming btn_clear_all is defined earlier in your code:
btn_clear_all.config(command=clear_allb)
btnsearch.config(command=search_products)
btn_generate.config(command=generate_bill)
# Main event loop
show_all_products()
window.mainloop()

```

7 Budget and Financial Plan

7.1 Cost Estimation

Hardware Costs

Item	Quantity	Cost (INR)	Total Cost (INR)
Computer/Laptop (Development)	1	45,000	45,000
Internet Connection	1	1,000/month	6,000 (for 6 months)
Printer (for billing, optional)	1	8,000	8,000

Hardware Cost Total: ₹59,000

Software Costs

Software	Cost (INR)
Python (Open-source)	Free
MySQL (Community Edition)	Free
Tkinter (Built-in with Python)	Free
NetBeans IDE (optional)	Free
OS (Windows/Linux)	Already installed (assumed)

Software Cost Total: ₹0

3. Development Costs

Development Activity	Estimated Cost (INR)
System Design (ERD, GUI, etc.)	10,000
Coding and Development	25,000
Testing and Debugging	8,000
Documentation Preparation	5,000

Development Cost Total: ₹48,000

4. Maintenance Costs

Maintenance Activity	Estimated Cost (INR)
System Updates (Yearly)	5,000
Bug Fixes and Enhancements	3,000

Annual Maintenance Cost: ₹8,000

7.2 Financial Planning

1. Budget Breakdown

Category	Estimated Cost (INR)
Hardware (PC, printer, etc.)	₹50,000
Development Cost	₹40,000
Maintenance (First Year)	₹10,000

Total Budget: ₹1,00,000

2. Funding Options

- **Self-Funding:** Use savings or business profits.
- **Bank Loan:** Apply for a small business or IT-specific loan.
- **Government Schemes:** Check for grants supporting digital transformation.

3. Estimated Benefits

- **Time Savings:** Faster billing and inventory management.
- **Cost Savings:** Reduce errors and manual work, saving ₹5,000/month.
- **Revenue Growth:** Better customer service may increase sales by ₹10,000/month.

Total Monthly Benefit: ₹15,000

Annual Benefit: ₹15,000 × 12 = ₹1,80,000

8 Future Enhancement

- **Integration with Barcode Scanning**
 - **Feature:** Add barcode scanner support for products to speed up billing and inventory updates.
 - **Benefit:** Reduces manual errors and improves efficiency in product management.
- **Online Ordering System**
 - **Feature:** Allow customers to browse inventory and place orders online.
 - **Benefit:** Expands your business reach and increases sales.
- **Advanced Sales Reporting**
 - **Feature:** Add features like monthly/quarterly sales trends, low-stock alerts, and profit analysis.
 - **Benefit:** Helps owners make informed decisions and optimize inventory management.
- **Mobile App Integration**
 - **Feature:** Develop a mobile app for owners and employees to manage the system remotely.
 - **Benefit:** Enables real-time inventory and sales tracking on the go.
- **Customer Management System**
 - **Feature:** Store customer information and purchase history.
 - **Benefit:** Helps with loyalty programs, personalized offers, and improved customer service.
- **Multi-Store Management**
 - **Feature:** Expand the system to handle multiple store locations.
 - **Benefit:** Centralized management of inventory and sales across all branches.
- **AI-Based Demand Prediction**
 - **Feature:** Use AI algorithms to predict demand for products based on past sales data.
 - **Benefit:** Optimizes inventory stocking and reduces wastage.
- **Integration with Payment Gateways**
 - **Feature:** Support multiple payment modes like UPI, credit/debit cards, and wallets.
 - **Benefit:** Provides seamless payment options for customers and better billing experience.
- **Employee Performance Analytics**
 - **Feature:** Track employee performance based on sales, attendance, and productivity.
 - **Benefit:** Enhances workforce management and rewards top performers.
- **Cloud-Based Data Storage**
 - **Feature:** Store data on the cloud instead of local servers.
 - **Benefit:** Ensures data security, scalability, and easy remote access.

9 Conclusion

The "Electronics Shop Inventory Management System" is a comprehensive solution tailored to streamline the operational needs of an electronics shop. By incorporating features such as employee management, supplier management, category and product management, billing, and sales reporting, the system enhances efficiency, reduces manual workload, and minimizes errors.

With its user-friendly interface and robust functionality, the system not only simplifies inventory management but also contributes to improved decision-making through detailed sales analysis. Designed with Python Tkinter and MySQL, it ensures reliability, scalability, and easy maintenance.

This system empowers shop owners and employees to focus on customer satisfaction and business growth. With future enhancements, such as online ordering, mobile app integration, and AI-based demand prediction, the system has the potential to evolve into a cutting-edge tool that meets the ever-changing demands of the retail industry.

Overall, the project delivers significant value by increasing operational efficiency, optimizing resource utilization, and driving profitability.

10 References

- **Python Documentation**
 - URL: <https://docs.python.org/3/>
 - Description: Official documentation for Python programming, used for implementing the backend logic of the system.
- **Tkinter Documentation**
 - URL: <https://docs.python.org/3/library/tkinter.html>
 - Description: Reference for designing the graphical user interface (GUI) of the system.
- **MySQL Documentation**
 - URL: <https://dev.mysql.com/doc/>
 - Description: Official documentation for MySQL, used for database design and management.
- **Stack Overflow**
 - URL: <https://stackoverflow.com/>
 - Description: Community-driven platform for resolving technical challenges during development.
- **W3Schools**
 - URL: <https://www.w3schools.com/>
 - Description: Resource for learning SQL queries, Python basics, and other web technologies.
- **GeeksforGeeks**
 - URL: <https://www.geeksforgeeks.org/>
 - Description: Educational platform used for understanding programming concepts and database operations.
- **Real Python**
 - URL: <https://realpython.com/>
 - Description: Comprehensive tutorials on Python programming, used for understanding Python best practices.
- **TutorialsPoint**
 - URL: <https://www.tutorialspoint.com/>
 - Description: Platform offering beginner-to-advanced tutorials on Python, Tkinter, and MySQL.